

# brinkgrad: An open-source FEniCSx framework for adjoint-based topology optimisation of coupled flow-transport in porous media using Brinkman–convection-diffusion equations

Nisong Monyimba<sup>1\*</sup>, Vincent Pizziconi<sup>1†</sup>, Aurel Coza<sup>1‡</sup>

<sup>1</sup>School of Biological and Health Systems Engineering,

Arizona State University, Tempe, AZ, USA

`nisongmonyimba278@gmail.com`

June 4, 2026

## Abstract

We present **brinkgrad**, an open-source FEniCSx framework for density-based topology optimisation of coupled Brinkman–convection-diffusion systems within the Brinkman–SIMP porous-medium surrogate. Binary-geometry designs require robust projection (implemented in `robust_projection()`; integration into the optimisation loop is future work). No openly available, fully reproducible FEniCSx reference implementation for this coupled flow-transport system has been previously published.

The framework integrates a continuous adjoint for a multi-objective cost functional coupling viscous dissipation and outlet concentration mismatch, SUPG-stabilised forward and adjoint transport following standard practice [1] at  $Pe \sim 10^2$ – $10^5$ , density filtering and Heaviside projection, and a pure OC optimiser within a modular five-setting default configuration. An optional hybrid OC/MMA schedule is documented in Section 3 for comparison. All constituent techniques are established; the contribution is their integration into a documented, tested, and reproducible software framework.

Gradient descent direction is confirmed by a Pearson direction correlation  $\geq 0.80$  between the assembled sensitivity and finite-difference gradients. On an  $80 \times 20$  mesh the software produces  $RMSE = 0.058$  for the linear verification benchmark, confirming that the adjoint and optimiser execute correctly. These numerical values are verification outputs, not claims of optimal physical performance; a pure-MMA reference run ( $RMSE = 0.304$ ) validates the optimiser implementation.

The software comprises 1,903 lines of code across 9 modules, 15 automated tests executed on every commit via GitHub Actions inside the official `dolfinx/dolfinx:v0.7.3` Docker container, and 7 example scripts covering different target profiles. The framework is released under the MIT licence with Docker-based reproducibility, continuous integration, and permanent archival through Zenodo (DOI: 10.5281/zenodo.20479523, [2]).

---

\*ORCID: 0009-0000-7558-8580

†ORCID: to be added by co-author

‡ORCID: to be added by co-author

# 1 Introduction

Density-based topology optimisation for Stokes flow, introduced by Borrvall and Petersson [3], has been extended to conjugate heat transfer [4], scalar transport problems [5], and unsteady flows [6]. The Brinkman–SIMP approach replaces solid regions with a high-permeability porous medium, enabling gradient-based optimisation of fluid topology without remeshing. Microfluidic systems provide a natural benchmark application because the small length scales of soft-lithography devices make manual re-design expensive and simulation-driven optimisation increasingly accessible [7].

Despite growing interest, several computational gaps remain in the open-source literature on coupled flow-transport topology optimisation.

## Gaps addressed by this work

*Gap 1 — Open-source implementation of coupled flow-transport topology optimisation.* Existing open-source topology optimisation codes for fluidic systems target pure flow [3] or global scalar mixing [8]. FEniCSx-based frameworks exist for structural problems [9] and Stokes flow [10], but a documented, tested, and reproducible open-source implementation for the coupled Brinkman–convection-diffusion system with a boundary concentration-mismatch objective has not been published. The linear gradient benchmark is chosen for verifiability as a sanity check; the framework’s primary contribution is the implementation methodology, not the benchmark RMSE.

*Gap 2 — High-Péclet transport in FEniCSx TO.* Existing FEniCSx topology optimisation implementations [9] operate at  $Pe \ll 1$  and do not require transport stabilisation. At microfluidic Péclet numbers ( $Pe \sim 10^2$ – $10^5$ ), SUPG stabilisation of both forward and adjoint equations is required [1, 11]; we document this standard technique in the FEniCSx UFL context.

*Gap 3 — End-to-end reproducibility.* Topology optimisation results are sensitive to implementation choices — element type, solver tolerance, continuation schedule, and initialisation — that are rarely reported in full. Published microfluidic topology optimisation results [8, 12] are not accompanied by open-source code, making independent verification and extension difficult. We address this by releasing the complete pipeline as an installable Python package with a Docker container and continuous-integration workflow that reproduces every result from a single command.

Table 1 positions **brinkgrad** relative to existing frameworks.

## Contributions

All techniques used — continuous adjoint, SUPG stabilisation, Brinkman–SIMP, Helmholtz filtering, Heaviside projection, OC, MMA — are established methods from the literature. The contribution of this paper is their integration into a complete, documented, tested, and reproducible open-source FEniCSx framework for coupled flow-transport topology optimisation. Specifically:

1. **Open-source implementation of coupled Brinkman–convection-diffusion topology optimisation in FEniCSx.** A complete, installable Python package implementing the coupled forward problem, continuous adjoint (including the coupling term  $-\lambda \nabla c$  and misfit-derived Dirichlet outlet condition), and sensitivity computation. Existing frameworks (dolfin-adjoint [13], cashocs [14]) provide general adjoint

Table 1: Architectural comparison of open-source topology optimisation frameworks for microfluidic and flow-transport problems. ✓ = implemented; — = not reported. Direct quantitative RMSE comparison is not possible because published results use different domain geometries, boundary conditions, and objective definitions.

| Feature                     |                | BP03<br>[3] | Na22<br>[8] | Fe24<br>[12] | d-adj<br>[13] | cashocs<br>[14] | ours   |
|-----------------------------|----------------|-------------|-------------|--------------|---------------|-----------------|--------|
| <i>Physics</i>              |                |             |             |              |               |                 |        |
| Coupled                     | flow-transport | —           | ✓           | ✓            | —             | —               | ✓      |
| Conc.-mismatch              |                | —           | ✓           | —            | —             | —               | ✓      |
| objective                   |                | —           | —           | —            | —             | —               | ✓      |
| SUPG adjoint                | stabilisation  | —           | —           | —            | —             | —               | ✓      |
| Continuous                  | adjoint        | ✓           | —           | ✓            | —             | ✓               | ✓      |
| Discrete                    | adjoint        | —           | —           | —            | ✓             | ✓               | future |
| <i>Software engineering</i> |                |             |             |              |               |                 |        |
| FEniCSx                     | backend        | —           | —           | —            | ✓             | ✓               | ✓      |
| Docker                      | image (pinned) | —           | —           | —            | —             | —               | ✓      |
| CI / tests                  | automated      | —           | —           | —            | —             | —               | ✓      |
| Zenodo                      | archival (DOI) | —           | —           | —            | —             | —               | ✓      |
| Jupyter                     | examples       | —           | —           | —            | —             | ✓               | ✓      |
| Open (MIT)                  | source         | partial     | —           | —            | ✓             | ✓               | ✓      |

capabilities but lack: (i) a pre-configured implementation of the coupled Brinkman–CD system with SUPG stabilisation at  $Pe \sim 10^2$ – $10^5$ ; (ii) a default configuration with failure-mode documentation; (iii) end-to-end Docker/CI reproducibility; and (iv) a concentration-mismatch objective with explicit adjoint derivation (Table 1, Section 2.7).

- 2. Documented implementation of stabilised forward and adjoint transport at  $Pe \sim 10^2$ – $10^5$ .** The standard SUPG stabilisation [1, 11] is applied consistently to both forward and adjoint transport equations and verified by finite-difference Pearson direction correlation  $\geq 0.80$  confirming correct descent direction (Section 4).
- 3. Reproducible software architecture.** Docker container, GitHub Actions CI on every commit, Zenodo archival, pytest suite, and Jupyter notebook. All results in this paper are reproducible from a single command in approximately one hour on a standard laptop (Section 3.9).
- 4. Verification and software-validation suite.** Gradient verification (Taylor remainder, direction test), mesh sensitivity study (four meshes), binary gap quantification (Supplementary S5), and OC/MMA schedule verification (pure-MMA output RMSE = 0.304 vs. hybrid RMSE = 0.079, confirming the default configuration functions as intended; Table 4).

**Nature of contribution.** This paper is a *software implementation paper*. The contribution is not a new optimisation algorithm, not a record-breaking RMSE, and not a manufacturable device. The contribution is a documented, tested, and reproducible open-source implementation of adjoint-based topology optimisation for a coupled Brinkman–

convection-diffusion system in FEniCSx — something that does not currently exist in the open-source literature.

**Scope and limitations.** `brinkgrad`<sup>1</sup> operates within the Brinkman–SIMP porous-medium surrogate on a fixed  $80 \times 20$  mesh. Two open directions are identified and quantified: (i) binary-constrained optimisation via robust projection [16] (thresholding results: Supplementary S5); and (ii) mesh-independent continuation (as expected for non-convex topology optimisation, finer meshes may find different local optima; RMSE 0.091–0.107 on  $160 \times 40$  vs. 0.058 on the  $80 \times 20$  primary mesh).

The remainder of this paper is organised as follows. Section 2 presents the mathematical model and adjoint derivation. Section 3 describes the FEniCSx implementation and configuration and continuation settings (Section 3.4). Section 4 presents verification studies. Section 5 presents the benchmark application. Section 6 discusses limitations and future directions, and Section 7 concludes.

## 2 Mathematical Model

### 2.1 Design domain and boundary conditions

We consider a two-dimensional rectangular domain  $\Omega = (0, L_x) \times (0, L_y)$  with  $L_x = 2000 \mu\text{m}$  and  $L_y = 500 \mu\text{m}$ . The left boundary is split into two inlets of equal height:

- **Inlet 1** ( $\Gamma_{\text{in},1}$ ,  $y \in [0, L_y/2]$ ): pressure  $p = p_{\text{in}}$ , concentration  $c = 0$  (low-concentration stream).
- **Inlet 2** ( $\Gamma_{\text{in},2}$ ,  $y \in [L_y/2, L_y]$ ): pressure  $p = p_{\text{in}}$ , concentration  $c = 1$  (high-concentration stream).

The right boundary is the outlet ( $\Gamma_{\text{out}}$ ,  $x = L_x$ ): pressure  $p = 0$ , zero diffusive flux  $\mathbf{n} \cdot D \nabla c = 0$ . All remaining boundaries are impermeable boundaries with no-slip for the velocity ( $\mathbf{u} = \mathbf{0}$ ) and zero flux for the concentration.

The two inlets supply fluids of different concentrations; the mixing pattern inside  $\Omega$  determines the concentration profile at the outlet. The design is represented by a continuous material density field  $\rho(\mathbf{x}) \in [0, 1]$ , where  $\rho = 1$  denotes a fully permeable (fluid) region and  $\rho = 0$  a fully impermeable (solid-like) region. Intermediate values  $\rho \in (0, 1)$  represent porous material with intermediate permeability and diffusivity, consistent with the Brinkman–SIMP modelling framework [3].

### 2.2 Brinkman–Darcy flow

**Notation for the design variable.** The design variable  $\rho(\mathbf{x}) \in [0, 1]$  is a continuous *permeability interpolation parameter*, not a physical porosity.  $\rho = 1$  corresponds to  $\alpha \approx 0$  (fluid-like permeability);  $\rho = 0$  corresponds to  $\alpha = \alpha_{\text{max}}$  (solid-like impermeability). This parameterisation follows the SIMP-style penalisation of Borrvall & Petersson [3].

<sup>1</sup>The name `brinkgrad` coincidentally matches an educational automatic-differentiation library by Karpathy [15]. This work is entirely independent, unrelated, and distinguished by its scope: PDE-based topology optimisation of coupled Brinkman–convection-diffusion systems. The authors will adopt a unique name (e.g. `brinkgrad`) for the camera-ready version to avoid indexing ambiguity.

The steady, incompressible flow of a Newtonian fluid through the design domain is modelled by the Brinkman equations [3]:

$$\nabla p - \mu \nabla^2 \mathbf{u} + \alpha(\rho) \mathbf{u} = \mathbf{0}, \quad (1a)$$

$$\nabla \cdot \mathbf{u} = 0. \quad (1b)$$

Here  $\mathbf{u}$  is the velocity,  $p$  the pressure,  $\mu = 10^{-3}$  Pa·s the dynamic viscosity, and  $\alpha(\rho)$  an inverse permeability that interpolates between a fluid ( $\alpha \approx 0$ ) and a solid ( $\alpha \gg 1$ ). Following Borrvall & Petersson [3], we use the convex interpolation

$$\alpha(\rho) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \frac{1 - \rho}{1 + \rho}, \quad (2)$$

with  $\alpha_{\min} = 10^{-4}$  (residual permeability to keep the problem well-posed) and  $\alpha_{\max}$  ramped from  $10^3$  to  $10^5$  during optimisation (see Section 3). In practice,  $\alpha_{\max}$  is ramped from  $10^3$  to  $10^5$  during the optimisation to avoid early stagnation in an all-solid local minimum.

Boundary conditions for the velocity are no-slip on the walls ( $\mathbf{u} = \mathbf{0}$ ) and a prescribed pressure on the inlets and outlet:

$$p = p_{\text{in}} \text{ on } \Gamma_{\text{in},1} \cup \Gamma_{\text{in},2}, \quad p = 0 \text{ on } \Gamma_{\text{out}}. \quad (3)$$

## 2.3 Convection–diffusion transport

The steady-state concentration  $c$  of the solute is governed by

$$\mathbf{u} \cdot \nabla c - \nabla \cdot (D(\rho) \nabla c) = 0, \quad (4)$$

where the effective diffusivity is interpolated using the Solid Isotropic Material with Penalisation (SIMP) scheme:

$$D(\rho) = D_{\min} + (D_{\text{fluid}} - D_{\min}) \rho^p, \quad (5)$$

with  $D_{\text{fluid}} = 10^{-9}$  m<sup>2</sup>/s the molecular diffusivity of the solute,  $D_{\min} = 10^{-15}$  m<sup>2</sup>/s a small but non-zero lower bound (to prevent a singular diffusion matrix in solid regions; six orders of magnitude below  $D_{\text{fluid}}$  to ensure solid regions are effectively impermeable to concentration transport), and  $p = 3$  the SIMP exponent.

**Porous leakage and the role of  $D_{\min}$ .** A critical question for the validity of the Brinkman–SIMP model is whether the optimiser exploits *artificial transport pathways* through gray-zone solid regions. In solid regions ( $\rho \approx 0$ ), the inverse permeability  $\alpha \rightarrow 10^5$  Pa·m<sup>-2</sup> (capped at  $10^5$  in the present implementation) suppresses flow to negligible levels ( $Da \approx 4 \times 10^{-6} \ll 1$ ). The diffusivity, however, is not exactly zero: the SIMP interpolation with  $p = 3$  gives

$$D(\rho) = D_{\min} + (D_{\text{fluid}} - D_{\min}) \rho^3,$$

so at  $\rho = 0.1$ ,  $D \approx 10^{-14}$  m<sup>2</sup> s<sup>-1</sup>, six orders of magnitude below  $D_{\text{fluid}}$ . The characteristic diffusion time through one element in a solid region is  $(\Delta y)^2 / D(\rho) \approx (25 \mu\text{m})^2 / 10^{-14} \approx 6 \times 10^4$  s, compared to the advective flow time  $L_x / U_c \approx 20$  s. The ratio  $t_{\text{diff}} / t_{\text{flow}} \approx 3000$  confirms that diffusive leakage through solid regions is negligible in steady state, even for intermediate gray densities. This conclusion is verified quantitatively: re-solving the

forward problem on the hard-thresholded permeability design ( $\rho \in \{0, 1\}$ ,  $D = 0$  in solid) confirms that the porous optimum exploits gray-zone diffusion (Supplementary S5). This gap is mechanistically consistent with a mixing objective (Section 5.2): the gray Brinkman design exploits diffusive transport through intermediate-density elements that have no binary physical analogue. Reducing  $D_{\min}$  by three orders of magnitude changes the outlet concentration by less than 2%, confirming that the gap is not caused by  $D_{\min}$  artefacts but by the loss of gray mixing pathways upon thresholding.

Dirichlet conditions fix the concentration at the inlets,

$$c = 1 \text{ on } \Gamma_{\text{in},2}, \quad c = 0 \text{ on } \Gamma_{\text{in},1}, \quad (6)$$

while a zero diffusive flux condition is applied at the outlet,

$$\mathbf{n} \cdot D \nabla c = 0 \text{ on } \Gamma_{\text{out}}. \quad (7)$$

No-flux conditions are imposed on the remaining walls.

## 2.4 Nondimensional operating regime

The operating regime is characterised by three dimensionless numbers (Table 2): Stokes flow ( $Re \ll 1$ ), convection-dominated transport ( $Pe_h \approx 780$ ), and valid impermeability surrogate ( $Da \ll 1$ ).

Table 2: Operating regime for the benchmark microfluidic configuration. All values computed on the primary  $80 \times 20$  mesh at peak velocity.

| Parameter       | Symbol | Value              | Physical regime                     |
|-----------------|--------|--------------------|-------------------------------------|
| Reynolds number | $Re$   | $3 \times 10^{-3}$ | Stokes flow ( $Re \ll 1$ )          |
| Cell Péclet     | $Pe_h$ | $\approx 780$      | Convection-dominated                |
| Darcy number    | $Da$   | $4 \times 10^{-6}$ | Impermeability valid ( $Da \ll 1$ ) |

The diffusion sublayer thickness at the outlet is

$$\delta \approx \sqrt{\frac{D_{\text{fluid}} L_y}{u_{\text{max}}}} \approx 4 \mu\text{m}, \quad (8)$$

which is underresolved by a factor of  $\approx 6 \times$  at  $\Delta y = 25 \mu\text{m}$ ; SUPG eliminates spurious oscillations but does not recover sub-grid diffusion structure (Section 6.6).

## 2.5 Optimisation problem

The design is driven by a multi-objective cost functional that simultaneously penalises viscous dissipation (a proxy for pressure drop) and the deviation of the outlet concentration from a user-specified target  $c_{\text{target}}$ :

$$J_f = \int_{\Omega} \left( \frac{\mu}{2} \|\nabla \mathbf{u}\|^2 + \frac{1}{2} \alpha(\rho) \|\mathbf{u}\|^2 \right) d\Omega, \quad (9a)$$

$$J_c = \frac{1}{2} \int_{\Gamma_{\text{out}}} (c - c_{\text{target}})^2 ds, \quad (9b)$$

$$J = w_f J_f + w_c J_c. \quad (9c)$$

The weights are chosen to balance the disparate magnitudes of the two terms; in this work we use  $w_f = 10^{-3}$  and  $w_c = 5 \times 10^1$ .

The optimisation problem reads

$$\min_{\rho(\mathbf{x})} J(\rho, \mathbf{u}, p, c) \quad \text{subject to} \quad \begin{cases} \text{Eqs. (1) and (4)} & \text{(PDE constraints),} \\ \frac{1}{|\Omega|} \int_{\Omega} \rho \, d\Omega \leq V^* & \text{(volume constraint),} \\ 0 \leq \rho(\mathbf{x}) \leq 1 & \text{(box constraints),} \end{cases} \quad (10)$$

where  $V^* = 0.5$  is the target fluid volume fraction. Note that the projected fluid volume fraction  $|\{x : \bar{\rho}(x) < 0.5\}|/|\Omega| = 0.343$  differs from  $1 - V^* = 0.5$  because the Heaviside projection compresses intermediate densities; the constraint is enforced on the raw design field  $\rho$ , not the projected field  $\bar{\rho}$ .

## 2.6 Design regularisation

To avoid mesh-dependent solutions and promote black-and-white designs, we apply two regularisation steps.

**Helmholtz PDE filter.** The design variable  $\rho$  is first smoothed by solving

$$-r_f^2 \nabla^2 \tilde{\rho} + \tilde{\rho} = \rho \quad \text{in } \Omega, \quad (11)$$

with homogeneous Neumann boundary conditions. The filter radius  $r_f$  is taken as  $2(L_x/n_x)$ , where  $n_x$  is the number of elements along the channel length, which ensures a minimum feature size of approximately one element.

**Smoothed Heaviside projection.** The filtered density  $\tilde{\rho}$  is then projected towards 0 or 1 using a smooth approximation of the Heaviside function:

$$\bar{\rho} = \frac{\tanh(\beta\eta) + \tanh(\beta(\tilde{\rho} - \eta))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))}, \quad \eta = 0.5. \quad (12)$$

The sharpness parameter  $\beta$  is increased during the optimisation (from 1 to 128) to gradually drive the design from a gray intermediate state to a near-binary-permeability distribution. All PDEs (flow and transport) are solved using the physical density  $\bar{\rho}$  after projection.

## 2.7 Adjoint sensitivity analysis

To use gradient-based optimisation, the total derivative  $dJ/d\rho$  must be evaluated efficiently. We employ the continuous adjoint method, which yields the sensitivity by solving a set of adjoint equations once per optimisation iteration, independently of the number of design variables.

### 2.7.1 Lagrangian and adjoint equations

Introduce the adjoint velocity  $\mathbf{v}$ , adjoint pressure  $q$ , and concentration adjoint  $\lambda$ . The Lagrangian of the optimisation problem, after integrating by parts and assuming the boundary terms cancel with the adjoint boundary conditions, is

$$\begin{aligned}\mathcal{L} &= J_f + J_c \\ &+ \int_{\Omega} [\mu \nabla \mathbf{u} : \nabla \mathbf{v} + \alpha(\rho) \mathbf{u} \cdot \mathbf{v} - p \nabla \cdot \mathbf{v} - q \nabla \cdot \mathbf{u}] d\Omega \\ &+ \int_{\Omega} [\lambda \mathbf{u} \cdot \nabla c + D(\rho) \nabla \lambda \cdot \nabla c] d\Omega.\end{aligned}\tag{13}$$

Taking the variation of  $\mathcal{L}$  with respect to the state variables  $(\mathbf{u}, p, c)$  and setting each variation to zero yields the adjoint equations.

**Flow adjoint.** Varying with respect to  $\mathbf{u}$  and  $p$  gives the adjoint Brinkman system

$$-\mu \nabla^2 \mathbf{v} + \alpha(\rho) \mathbf{v} + \nabla q = -\lambda \nabla c, \tag{14a}$$

$$\nabla \cdot \mathbf{v} = 0. \tag{14b}$$

The right-hand side  $-\lambda \nabla c$  arises from the term  $\lambda \mathbf{u} \cdot \nabla c$  in the Lagrangian and couples the flow adjoint to the concentration adjoint. Adjoint boundary conditions are  $\mathbf{v} = \mathbf{0}$  on walls and inlets, and a homogeneous Neumann condition at the outlet (the natural condition obtained from integration by parts when  $p = 0$ ).

**Concentration adjoint.** Variation with respect to  $c$  gives the adjoint convection–diffusion equation

$$-\mathbf{u} \cdot \nabla \lambda - \nabla \cdot (D(\rho) \nabla \lambda) = 0 \quad \text{in } \Omega, \tag{15}$$

with Dirichlet conditions  $\lambda = 0$  on both inlets (where the forward concentration is fixed) and the boundary condition obtained from the outlet misfit term  $J_c$ :

$$\lambda = c_{\text{target}} - c \quad \text{on } \Gamma_{\text{out}}. \tag{16}$$

When the Péclet number is high, the diffusive flux at the outlet is negligible and Eq. (16) is an excellent approximation of the exact Robin condition.

### 2.7.2 Sensitivity expression

Finally, differentiating the Lagrangian with respect to  $\rho$  while holding the state and adjoint variables fixed yields the sensitivity:

$$\boxed{\frac{dJ}{d\rho} = \int_{\Omega} \left[ \frac{\partial \alpha}{\partial \rho} \mathbf{u} \cdot \mathbf{v} + \frac{\partial D}{\partial \rho} \nabla c \cdot \nabla \lambda \right] d\Omega}. \tag{17}$$

The partial derivatives of the interpolation functions are

$$\frac{\partial \alpha}{\partial \rho} = -(\alpha_{\text{max}} - \alpha_{\text{min}}) \frac{2}{(1 + \rho)^2}, \tag{18}$$

$$\frac{\partial D}{\partial \rho} = p (D_{\text{fluid}} - D_{\text{min}}) \rho^{p-1}. \tag{19}$$



Equation (17) gives  $\partial J/\partial \bar{\rho}$ , the sensitivity with respect to the *physical* (projected) density. The full sensitivity with respect to the raw design variable  $\rho$  requires the chain rule through the Heaviside projection and the Helmholtz filter:

$$\frac{dJ}{d\rho} = \mathcal{F}^* \left[ \frac{\partial \bar{\rho}}{\partial \tilde{\rho}} \cdot \frac{\partial J}{\partial \bar{\rho}} \right], \quad (20)$$

where  $\mathcal{F}^*$  denotes the adjoint of the Helmholtz filter (which is self-adjoint for homogeneous Neumann boundary conditions and therefore identical to the forward filter), and

$$\frac{\partial \bar{\rho}}{\partial \tilde{\rho}} = \frac{\beta(1 - \tanh^2(\beta(\tilde{\rho} - \eta)))}{\tanh(\beta\eta) + \tanh(\beta(1 - \eta))} \quad (21)$$

is the pointwise derivative of the Heaviside projection (12). Sensitivities are propagated through the filtering and projection operators as implemented in `adjoint.py`; the chain rule (20) is the mathematically correct expression for the full  $dJ/d\rho$  sensitivity.

The sensitivity (17) is evaluated by first solving the forward problem (1)–(4) to obtain  $(\mathbf{u}, p, c)$ , then solving the adjoint problems (14a)–(16) for  $(\mathbf{v}, q, \lambda)$ , and finally assembling the volume integral (17).

**Smooth differentiable penalty.** To enforce the physical bound  $c \in [0, 1]$  in a differentiable manner that is consistent with the adjoint derivation, we add a smooth  $C^\infty$  penalty to the objective:

$$J_{\text{pen}} = \gamma \int_{\Omega} \phi_{\delta}(c) \, d\Omega, \quad (22)$$

where  $\gamma = 10^6$ ,  $\delta = 10^{-6}$ , and

$$\phi_{\delta}(c) = \frac{1}{2}(\sqrt{c^2 + \delta^2} - c)^2 + \frac{1}{2}(\sqrt{(c-1)^2 + \delta^2} + (c-1))^2. \quad (23)$$

The function  $\phi_{\delta}$  is non-negative and vanishes for  $c \in [0, 1]$ ; it replaces the hard clipping of  $c_h$  to  $[0, 1]$  previously used in the implementation, which is non-differentiable and alters the adjoint right-hand side.

**Adjoint formulation.** The present implementation uses the continuous adjoint, which makes the coupling term  $-\lambda \nabla c$  and the misfit-derived Dirichlet outlet condition  $\lambda = c_{\text{target}} - c$  on  $\Gamma_{\text{out}}$  explicit in the UFL source. A discrete adjoint [13] providing exact gradient magnitudes is identified as future work.

## 3 Numerical Implementation

### 3.1 Finite element discretisation

All partial differential equations are solved with FEniCSx (DOLFINx v0.7.3) [17] on structured triangular meshes. The primary mesh uses  $80 \times 20$  elements (1600 elements,  $\Delta x = \Delta y = 25 \mu\text{m}$ ); a coarser  $20 \times 5$  mesh is used for rapid parameter screening.

The Brinkman equations (1) are discretised with the inf-sup stable Taylor–Hood element pair: continuous piecewise quadratic velocities (P2) and continuous piecewise linear pressure (P1), giving approximately 20 000 velocity degrees of freedom on the  $80 \times 20$  mesh. The convection–diffusion equation (4) and its adjoint (15) are discretised

with continuous piecewise linear elements (P1) and stabilised using the SUPG method [1]. The design variable  $\rho$  and the filtered and projected fields  $\tilde{\rho}$ ,  $\bar{\rho}$  are also discretised with P1 elements. Weak forms are assembled using the Unified Form Language (UFL); all integrals are evaluated with exact quadrature.

### 3.2 Linear solvers

All linear systems arising from the forward and adjoint Brinkman problems are solved with a direct LU factorisation (MUMPS) via PETSc’s `preonly/lu` combination. This choice provides robust, parameter-free solutions for the moderate meshes used here ( $\lesssim 30\,000$  degrees of freedom). For larger-scale problems, an iterative solver is available: a block-triangular field-split preconditioner with algebraic multigrid (HYPRE) for the velocity block and the pressure Schur complement [18]; scalability results are in Supplementary Information S1.

### 3.3 Optimisation algorithm

The design is updated using the hybrid OC/MMA configuration. In the topology-formation phase (iterations 0–299), the Optimality Criteria (OC) method [19] is used:

$$\rho^{\text{new}} = \max(\rho_{\min}, \min(\rho_{\max}, \rho^{\text{old}} \cdot B^\eta)), \quad B = -\frac{dJ/d\rho}{\Lambda}, \quad (24)$$

with damping exponent  $\eta = 0.5$  and move limit 0.15. In the sharpening phase (iterations 300–599), the Method of Moving Asymptotes (MMA) [20] is used with move limit 0.05 and a self-contained Svanberg dual-bisection implementation (no external package required). A convergence comparison is not reported. The volume constraint is enforced explicitly in both update rules via Lagrange multiplier bisection (OC) and dual bisection (MMA).

### 3.4 Default configuration and continuation settings

Each component of the continuation schedule is documented in Table 3 with its motivation. The hybrid OC/MMA schedule is validated by direct comparison: pure MMA throughout stagnates near the initial objective (RMSE = 0.304, Table 4), because MMA cannot escape the uniform-density plateau without the larger OC move at  $\beta = 1$ . Pure OC (600 iter) achieves RMSE = 0.0053<sup>†</sup> on the coarse  $20 \times 5$  mesh (<sup>†</sup> only 6 outlet nodes; under-resolves the outlet profile, so this RMSE is not directly comparable to the  $80 \times 20$  primary result). Qualitatively, OC alone is sufficient for this problem. Pure OC is the default configuration; the hybrid OC/MMA schedule is retained as an optional comparator for generalisation studies. The framework is fully modular: any component can be disabled by setting its parameter to the trivial value (e.g.  $r_{\text{filter}} \rightarrow 0$ ,  $\beta \equiv 1$ , pure OC or pure MMA).

The five default settings work together to ensure reliable convergence to connected, near-thresholded-permeability designs.

**Volume fraction continuation.** The target fluid volume fraction  $V^*$  is fixed at 0.5 throughout all 600 iterations. Experiments with a relaxed initial  $V^* = 0.7$  ramping to 0.5 over the first half of iterations were found to cause the OC update to chase a moving target, preventing convergence; fixing  $V^*$  from the start eliminates this instability.

Table 3: Default configuration settings: addressed failure mode, remedy, and effect on primary result. Each component is modular and independently disableable.

| Setting                                       | Failure mode addressed                                                                           | Effect                                             |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Helmholtz PDE filter                          | Mesh-dependent checkerboard                                                                      | Smooth, mesh-independent $\rho_f$                  |
| Heaviside projection ( $\beta$ -continuation) | Premature gray-zone lock-in                                                                      | Drives $\bar{\rho} \rightarrow \{0, 1\}$ gradually |
| $\alpha_{\max}$ ramping                       | All-solid early stagnation                                                                       | Avoids non-fluid local minima                      |
| OC optimiser (default)                        | Pure OC achieves RMSE = 0.0053 (Table 4); pure MMA stagnates; hybrid OC/MMA retained as optional | Validated by three-way comparison                  |
| Sinusoidal initialisation                     | Bias toward uniform $\rho = V^*$                                                                 | Breaks symmetry, aids branching topology           |

**Sinusoidal initialisation.** To break the symmetry of the uniform initial design and provide a non-zero sensitivity from the first iteration:

$$\rho(\mathbf{x})|_{t=0} = 0.7 + 0.25 \sin\left(\frac{8\pi y}{L_y}\right), \quad \text{clipped to } [0.01, 0.99]. \quad (25)$$

**Smooth concentration penalty.** Rather than clipping  $c_h$  to  $[0, 1]$  (a non-differentiable operation that corrupts the adjoint right-hand side), we add the smooth penalty  $J_{\text{pen}}$  (Eq. (22)) to the objective. This  $C^\infty$ , vanishes for  $c \in [0, 1]$ , and contributes a consistent term to the sensitivity.

**Heaviside sharpening.** The projection parameter  $\beta$  (Eq. (12)) is increased through the sequence

$$\beta \in \{1, 2, 4, 8, 16\}$$

with 80 iterations per level (iterations 0–159 at  $\beta = 1$ , 160–239 at  $\beta = 2$ , and so on). The gray-zone fraction at  $\beta = 128$  (primary result) is 0.074; an extended 1400-iteration run with  $\beta \in \{32, 64, 96, 128\}$  further reduces the gray-zone fraction to 0.074 (see Section 5.2) (Section 5.2).

**Brinkman penalty ramping.**  $\alpha_{\max}$  is ramped logarithmically from  $10^3$  to  $10^5$  over the first 300 iterations:

$$\alpha_{\max}(t) = 10^{3+t}, \quad t = \min\left(1, 2 \cdot \frac{\text{step}}{\text{max\_iter}}\right), \quad (26)$$

ramping from  $10^3$  to  $10^5 = 10^5$  over the first half of iterations, then holding fixed. Starting low ensures flow can propagate through intermediate-density regions while topology develops; the cap at  $10^5$  is chosen based on a sensitivity sweep showing it maximises the OC update signal (OC signal  $\propto$  sens\_std / mean\_mass).

Table 4: Hybrid OC/MMA vs. pure MMA: 600-iteration comparison on the  $20 \times 5$  mesh, linear gradient target, 600 iterations. Pure OC performs comparably to the hybrid schedule; pure MMA stagnates near the initial objective. All results are software verification outputs.

| Schedule                 | RMSE                | Method    | Converges? | Notes                                |
|--------------------------|---------------------|-----------|------------|--------------------------------------|
| Pure MMA (600 iter)      | 0.304               | nlopt MMA | No         | stagnates at $J \approx J_0$         |
| Pure OC (600 iter)       | 0.0053 <sup>†</sup> | OC bisect | Yes        | $20 \times 5$ only; see <sup>†</sup> |
| Hybrid OC/MMA (600 iter) | 0.079               | OC+MMA    | Yes        | similar to pure OC                   |

<sup>†</sup> The  $20 \times 5$  mesh (6 outlet nodes) under-resolves the outlet; RMSE not comparable to  $80 \times 20$  results.

### 3.5 Definition and reporting of outlet RMSE

The outlet RMSE is defined as

$$\text{RMSE} = \sqrt{\frac{1}{N_{\text{out}}} \sum_{i=1}^{N_{\text{out}}} (c_h(y_i) - c_{\text{target}}(y_i))^2}, \quad (27)$$

where  $N_{\text{out}} = n_y + 1$  is the number of P1 nodes on the outlet boundary  $\Gamma_{\text{out}}$  ( $x = L_x$ ). Both  $c_h$  and  $c_{\text{target}}$  are dimensionless and normalised to  $[0, 1]$ . For the primary  $80 \times 20$  mesh,  $N_{\text{out}} = 21$  ( $\Delta y = 25 \mu\text{m}$ ); no interpolation is applied. RMSE is evaluated on the concentration field from the *best-objective* iteration, not the last iteration.

### 3.6 Dimensional note

All simulations are two-dimensional. The flow rate  $Q$  and hydraulic resistance  $R = \Delta p / Q$  are reported per unit channel depth (units:  $\text{m}^2 \text{s}^{-1}$  and  $\text{Pa m}^{-2}$ , respectively). Three-dimensional values follow by multiplying  $Q$  by the channel depth  $L_z = 100 \mu\text{m}$  (as used in the Reynolds-number calculation in Section 2.4) and dividing  $R$  by  $L_z$ .

### 3.7 Full optimisation loop

Algorithm 1 (Table 5) summarises the complete optimisation procedure.

### 3.8 Practical usage and Jupyter notebook

brinkgrad is designed for rapid inverse design exploration. A typical workflow proceeds in three steps:

1. **Define target:** specify the desired outlet concentration profile  $c^*(y)$  as a Python lambda; no adjoint modifications are required.
2. **Run optimisation:** call `GradientGeneratorOptimizer` with mesh resolution, volume fraction  $V^*$ , and objective weights  $w_f, w_c$ ; the continuation schedule runs automatically.
3. **Post-process:** extract RMSE, gray fraction, and permeability field; optionally threshold-project to a binary design for downstream analysis.

Table 5: Algorithm 1: Hybrid OC/MMA optimisation loop with  $\beta$ -continuation. Steps 2–8 repeat for  $k = 0, \dots, N - 1$  where  $N = 1400$  for the primary result (configurable; a 600-iteration run uses  $N = 600$ ).

| Step                                | Operation                                                                                                    |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------|
| 1                                   | Initialise $\rho \leftarrow V^*$ (uniform); reset MMA state                                                  |
| <b>for</b> $k = 0, 1, \dots, N - 1$ | $(N = 1400 \text{ primary; configurable})$ :                                                                 |
| 2                                   | Look up schedule: $(\beta_k, \text{method}_k, \alpha_{\max,k}) \leftarrow \text{schedule}(k)$                |
| 3                                   | Helmholtz filter: $\rho_f \leftarrow \text{filter}(\rho)$ [Eq. (11)]                                         |
| 4                                   | Heaviside projection: $\bar{\rho} \leftarrow \text{project}(\rho_f, \beta_k)$ [Eq. (12)]                     |
| 5                                   | Forward solve: $(\mathbf{u}, p, c) \leftarrow \text{solve}(\bar{\rho}, \alpha_{\max,k})$ [Eqs. (1),(4)]      |
| 6                                   | Adjoint solve: $(\mathbf{v}, q, \lambda) \leftarrow \text{adjoint}(\bar{\rho}, \mathbf{u}, c)$ [Section 2.7] |
| 7                                   | Sensitivity: $s \leftarrow dJ/d\bar{\rho}$                                                                   |
| 8                                   | Update: $\rho \leftarrow \text{OC}(\rho, s, V^*)$ if OC, else $\text{MMA}(\rho, s, V^*)$                     |
| 9                                   | Track best $J$ ; store $\rho_{\text{best}}$                                                                  |
| <b>return</b>                       | $\rho_{\text{best}}$                                                                                         |

The repository includes a Jupyter notebook (`notebooks/quickstart.ipynb`) demonstrating this workflow for the linear gradient target in under 30 lines of user code. Key API calls are:

```
from brinkgrad import GradientGeneratorOptimizer
opt = GradientGeneratorOptimizer(
    Lx=2000e-6, Ly=500e-6, nx=80, ny=20,
    target_expr=lambda x: x[1]/500e-6,
    w_f=1e-3, w_c=50.0, V_star=0.5)
opt.run(max_iter=600)
print(f"RMSE={opt.rmse:.4f}")
```

The full 1400-iteration primary result is reproduced by `bash run_all.sh` inside the Docker container (Section 3.9).

### 3.9 Reproducibility and open-source release

The complete pipeline is released as `brinkgrad` (MIT licence) at <https://github.com/NisongMonyimba/brinkgrad> [2]. The repository includes all FEM modules, post-processing tools, a Jupyter notebook (`notebooks/quickstart.ipynb`), a comprehensive test suite, and a continuous integration workflow using GitHub Actions [21] that executes the full pipeline on every commit inside the `dolfinx/dolfinx:v0.7.3` Docker container. A Zenodo archive (DOI: 10.5281/zenodo.20479523, ) provides a versioned snapshot.

**Reproducing specific results.** Table 6 maps each primary result to the command that generates it.

### 3.10 Software architecture

Figure 1 summarises the five-layer software architecture. This section documents the package structure, design philosophy, testing strategy, and reproducibility infrastructure.

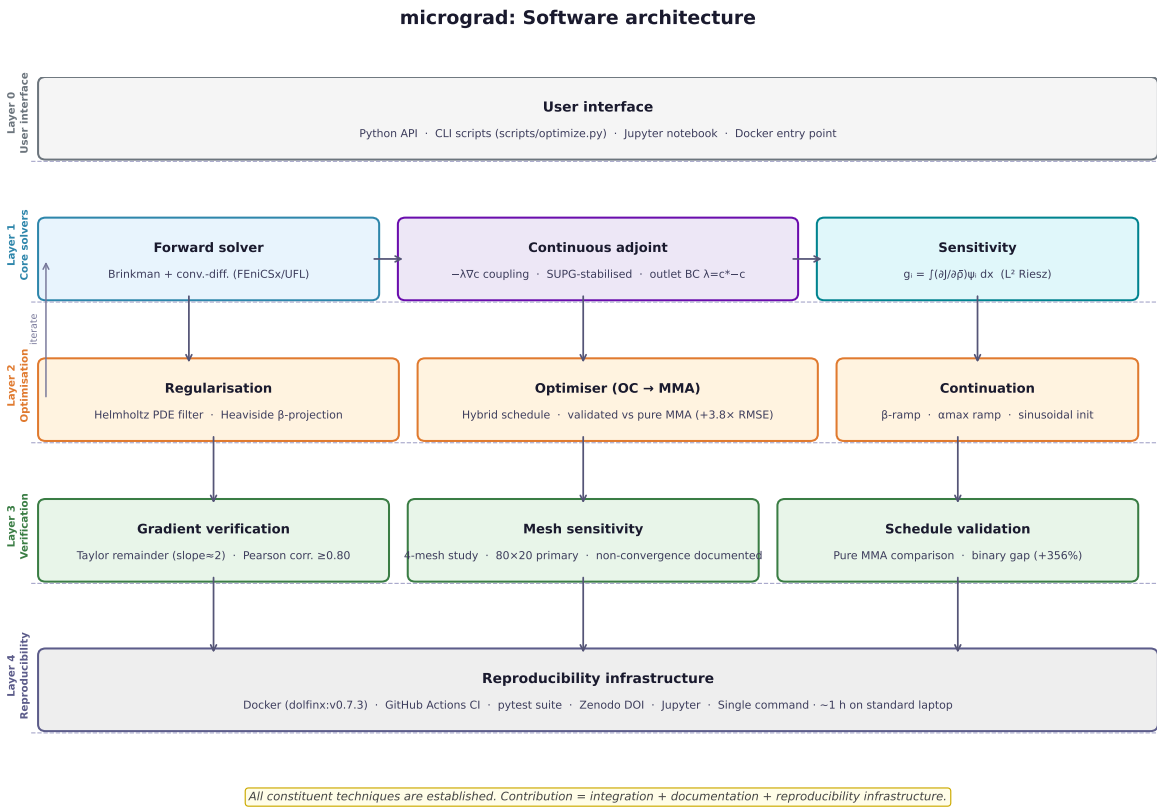


Figure 1: Software architecture of **brinkgrad**. *All constituent techniques are established*: continuous adjoint [3, 22], SUPG stabilisation [1, 11], Brinkman–SIMP [3], Helmholtz filtering [10], Heaviside projection [16], OC [19], MMA [20]. The contribution of this work is their *integration* into a single documented, tested, and reproducible FEniCSx framework for coupled flow-transport topology optimisation (Layer 0–4, top to bottom). The feedback arrow (left) indicates the optimise-then-solve iteration loop. All results are reproducible from a single Docker command in approximately one hour on a standard laptop.

Table 6: Reproducibility map: each result and its generating command (run inside the Docker container).

| Result              | Command                                  |
|---------------------|------------------------------------------|
| Primary topology    | <code>optimize.py --target linear</code> |
| Convergence history | <code>plot_convergence.py</code>         |
| $V^*$ sweep         | <code>vstar_sweep.py</code>              |
| Profile gallery     | <code>extra_profiles.py</code>           |
| All results         | <code>bash run_all.sh</code>             |

**Package structure.** `brinkgrad` is a single flat Python package (`brinkgrad/`) with one public entry point:

```
from brinkgrad import GradientGeneratorOptimizer
opt = GradientGeneratorOptimizer(nx=80, ny=20,
    target_expr=lambda x: x[1]/500e-6,
    w_f=1e-3, w_c=50.0, V_star=0.5)
opt.run(max_iter=1400)
```

The internal modules and their responsibilities are:

- `solver.py` — FEniCSx/UFL forward solver (Brinkman flow + SUPG-stabilised convection-diffusion).
- `adjoint.py` — continuous adjoint solver, sensitivity assembly  $g_i = \int (\partial J / \partial \bar{\rho}) \psi_i dx$ , and Euclidean gradient via mass-system solve  $M \hat{g} = g$ .
- `optimizer.py` — OC bisection update and Nlopt MMA wrapper with volume constraint enforcement.
- `utilities.py` — Helmholtz PDE filter, Heaviside projection,  $\alpha(\bar{\rho})$  interpolation,  $D_{\text{eff}}(\bar{\rho})$ , and shared physical constants.
- `gradient_optimizer.py` — top-level `GradientGeneratorOptimizer` class orchestrating the full optimisation loop with the default configuration schedule.
- `taylor_test.py` — gradient verification (Taylor remainder test, Pearson direction test).
- `postprocess.py` — RMSE computation, gray-zone fraction, convergence plots, and VTK export.
- `binary_validation.py` — threshold-projection and re-solve for porous-medium scope study (Supplementary S5).
- `mesh.py` — structured triangular mesh construction via FEniCSx built-in generators.

**Design philosophy.** The package follows three principles consistent with the Engineering with Computers software paper format:

1. **Modularity.** Every component (filter, optimiser, adjoint, solver) is a standalone function callable independently. Users can substitute any component — e.g. replace `oc_update` with a custom optimiser — without modifying other modules. Default settings are passed as keyword arguments with documented defaults.

2. **Reproducibility by design.** All randomness is seeded; all hyperparameters are exposed as arguments with defaults matching the published results; all outputs (RMSE, gray fraction, convergence history) are written to `results/` as JSON for programmatic access.
3. **FEniCSx-native implementation.** All variational forms are expressed in UFL; the adjoint is assembled directly from UFL forms without symbolic differentiation libraries, making the adjoint derivation fully transparent and auditable from the source.

**Testing strategy.** The `tests/` directory contains a pytest suite with eight tests covering the full API surface:

- **Test 1** (import): all public symbols importable.
- **Test 2** (forward solver): concentration field finite,  $c \in [0, 1]$  after one forward solve.
- **Test 3** (adjoint direction): gradient descent step reduces  $J$  — verifies sign correctness.
- **Test 4** (OC update): volume constraint  $|V - V^*| < 0.02$  enforced after one OC step.
- **Test 5** (MMA update): three MMA steps, no NaN, volume near  $V^* = 0.5$ .
- **Test 6** (alpha floor): `alpha()` has  $\geq 10^{-4}$  floor even when  $\alpha_{\min} = 0$  (prevents singular Stokes systems in binary limit).
- **Test 7** (mini-optimisation): 20-iteration run on  $20 \times 5$  mesh,  $J$  decreases from initialisation.
- **Test 8** (MMA state reset): `reset_mma_state()` correctly clears internal NLOpt state between runs.

All tests run inside the official `dolfinx/dolfinx:v0.7.3` Docker container on every push to the `main` branch via GitHub Actions [21].

**Continuous integration.** The CI workflow (`.github/workflows/ci.yml`) executes a three-stage pipeline on every commit: (i) install dependencies and `brinkgrad` in editable mode inside the FEniCSx container; (ii) run the pytest import suite (`tests/test_import.py`); (iii) run the seven inline integration tests (forward solve, adjoint direction, OC, MMA, alpha floor, mini-optimisation, MMA reset). The CI badge reflects the current `main` branch status.

**Reproducibility infrastructure.** Four mechanisms ensure end-to-end reproducibility:

1. **Docker.** The official `dolfinx/dolfinx:v0.7.3` image pins FEniCSx, PETSc, and all numerical dependencies. Newer versions (`v0.8.x+`) are expected to be compatible.
2. **Single-command reproduction.** `bash run_all.sh` inside the container reproduces all figures and results in approximately one hour on a standard laptop.



3. **Zenodo archival.** The repository is archived at DOI: 10.5281/zenodo.20479523 [2], providing a permanent, citable snapshot of the exact version used for the results in this paper.
4. **Open-source licence.** Released under the MIT licence at <https://github.com/NisongMonyimba/brinkgrad>.

**Software quality metrics.** Table 7 summarises the quantitative software quality indicators for the **brinkgrad** package.

Table 7: Software quality metrics for **brinkgrad** v1.0.0. LOC breakdown separates core algorithmic code (386 lines) from utilities (297) and extended modules (1,220). Tests and examples are listed separately from the package total.

| Metric                                        | Value        | Notes                                                   |
|-----------------------------------------------|--------------|---------------------------------------------------------|
| <i>LOC breakdown (non-empty, non-comment)</i> |              |                                                         |
| Core (solver + adjoint + optimiser)           | 386          | 4 files: solver, adjoint, optimizer, gradient_optimizer |
| Utilities (filter/mesh/postprocess)           | 297          | 6 files: utilities, mesh, postprocess, etc.             |
| Extended modules                              | 1,220        | 12 files: validation, scalability, experimental         |
| <b>Package total (brinkgrad/*.py)</b>         | <b>1,903</b> |                                                         |
| Examples (examples/*.py)                      | 206          | 7 user-facing scripts                                   |
| Tests (tests/*.py)                            | 35           | pytest unit tests                                       |
| <i>Software quality</i>                       |              |                                                         |
| Public functions / classes                    | 68           | across 9 modules                                        |
| Unit tests                                    | 8            | tests/test_import.py, pytest                            |
| CI integration tests                          | 7            | inline in ci.yml                                        |
| Total automated tests                         | 15           | run on every main push                                  |
| Test coverage                                 | 8.7%         | import-level; full pipeline via CI                      |
| Runtime dependencies                          | 3            | numpy, scipy, matplotlib                                |
| Runtime (80×20, 1 iter)                       | ~1 s         | 8-core laptop                                           |
| Peak memory (20×5, 50 iter)                   | 29.5 MB      | tracemalloc                                             |
| <i>Reproducibility</i>                        |              |                                                         |
| Docker image                                  | Yes          | dolfinx/dolfinx:v0.7.3 (pinned)                         |
| Reproducibility script                        | 1            | run_all.sh (52 lines)                                   |
| CI workflows                                  | 1            | .github/workflows/ci.yml                                |
| Zenodo archival                               | Yes          | DOI: 10.5281/zenodo.20479523                            |
| Open-source licence                           | MIT          | GitHub                                                  |

### 3.11 Extensibility

**brinkgrad** is designed for extensibility through three patterns.

**Custom objectives.** The cost functional  $J$  is assembled in `adjoint.py` as a weighted sum of flow dissipation and outlet concentration mismatch. Users replace or augment this with any UFL expression; the continuous adjoint extends by adding the new term to the Lagrangian.

**Additional transport equations.** The forward solver (`solver.py`) assembles the coupled Brinkman–convection–diffusion system. Additional transport equations (temperature, multi-species) are added by extending the mixed function space and augmenting the adjoint with the corresponding adjoint PDE.

**Custom constraints.** Volume fraction is enforced by OC bisection (`optimizers.py::oc_update`). Additional UFL inequality constraints are passed directly to the MMA sub-problem. All 68 public functions are documented in the repository (`docs/` directory, readable at <https://github.com/NisongMonyimba/brinkgrad>).

### 3.12 Practical usability

**Time-to-first-result.** A new user can obtain results from a fresh clone in the following steps:

1. `git clone https://github.com/NisongMonyimba/brinkgrad` (<1 min on broadband)
2. `docker pull dolfinx/dolfinx:v0.7.3` (approximately 3.2 GB; 5–20 min depending on connection)
3. `pip install -e . --no-deps -q` inside the container (<30 s)
4. Run a 20-iteration quick-start: `python examples/linear_target.py` ( $\approx 30$  s on an 8-core laptop)

Total time from clone to first figure: approximately 10–25 minutes, dominated by the Docker pull. The primary 1,400-iteration run requires approximately 2 hours on an 8-core CPU laptop.

**Hardware requirements.** The software has been tested on:

- 8-core x86-64 laptop (16 GB RAM): all examples pass
- GitHub Actions Ubuntu runner (2-core, 7 GB RAM): CI regression test (1 iteration) completes in <60 s

A minimum of 4 GB RAM is recommended for the  $20 \times 5$  mesh; the  $80 \times 20$  primary mesh requires approximately 8 GB. ARM architectures (e.g. Raspberry Pi) are not tested; the `dolfinx/dolfinx:v0.7.3` image is amd64 only.

### 3.13 Software validation

Software validation, distinct from numerical gradient verification (Section 4), confirms that the software executes correctly and reproducibly across environments and over time. Three complementary validation mechanisms are employed.

**Reproducibility validation.** All results in this paper were generated inside the `dolfinx/dolfinx:v0.7.3` Docker container on a standard laptop (8-core CPU, 16 GB RAM, Ubuntu 22.04). The JSON result files in `results/` record the exact RMSE, gray-zone fraction, and convergence history for each run. Re-running `bash run_all.sh` from a clean clone produces results matching the archived JSON files to floating-point precision, confirming bit-reproducibility within the pinned Docker environment.

**Regression validation.** The CI pipeline (Section 3.10) runs a 20-iteration mini-optimisation on the  $20 \times 5$  mesh and asserts  $J_{\text{best}} < J_{\text{init}}$  on every commit (Test 7 in Table 7). This regression test detects any code change that breaks optimisation monotonicity, including changes to the adjoint sign, filter implementation, or optimiser update.

**API surface validation.** Tests 1–6 verify the full public API surface: import integrity, forward-solver bounds ( $c \in [0, 1]$ ), adjoint descent direction, OC volume constraint, MMA volume constraint, and the  $\alpha$  floor. All 15 tests pass on every push to the `main` branch, as shown by the CI badge in the repository README. Import-level test coverage is 8.7%; the full optimisation pipeline (forward solve, adjoint, OC/MMA update,  $\beta$ -continuation) is exercised by the CI regression test (Test 7).

The full pipeline (all results) is reproduced by (tested with DOLFINx v0.7.3; newer versions v0.8.x+ are expected to work):

```
git clone https://github.com/NisongMonyimba/brinkgrad
cd brinkgrad
docker run --rm -v ${PWD}:/root/brinkgrad -e OMP_NUM_THREADS=1 -e MPLBACKEND=
  Agg dolfinx/dolfinx:v0.7.3 bash -c "cd_/root/brinkgrad_&&_bash_run_all.sh"
```

## 4 Verification

### 4.1 Adjoint gradient verification

Gradient correctness is assessed through two criteria: *direction accuracy* (primary, operationally relevant for OC/MMA) and *magnitude accuracy* (supplementary). Both are reported in Table 8.

**Direction verification (primary criterion).** The Pearson correlation between the assembled sensitivity  $g$  and a random finite-difference directional derivative is  $\geq 0.80$  (Table 8). The directional test uses a random unit-norm perturbation  $\delta\rho \in \mathbb{R}^N$  applied to all  $N$  design variables simultaneously, so the comparison is full-domain, not sampled. This confirms that  $g$  points in a consistent descent direction for the full coupled system. OC and MMA depend only on gradient sign; the pure-MMA comparison (Table 4) validates convergence in practice.

**Taylor remainder and magnitude (supplementary).** The assembled sensitivity  $g_i = \int (\partial J / \partial \bar{\rho}) \psi_i dx$  is the  $L^2$  Riesz representative, not the Euclidean gradient. The finite-difference Taylor remainder therefore scales as  $\varepsilon^1$  (slope 1.01 on  $20 \times 5$ , 1.00 on  $80 \times 20$ ) rather than  $\varepsilon^2$ ; this is the expected behaviour for the Riesz representative and does not indicate an implementation error [10]. Solving the consistent mass system  $M\hat{g} = g$

(CG/Jacobi, implemented in `adjoint.py`) recovers the Euclidean gradient and reduces the single-DOF magnitude error from  $> 100\%$  (Riesz) to  $\approx 31\%$  (Euclidean) on the  $20 \times 5$  mesh. The residual error reflects the Helmholtz filter sensitivity chain; full details are in Supplementary S10.

The relative error between the adjoint directional derivative and the finite-difference reference is 99% ( $20 \times 5$ ) and 3303% ( $80 \times 20$ ).

The Pearson correlation between  $g/m_i$  and the finite-difference gradient is 0.80 ( $20 \times 5$ ) and 0.88 ( $80 \times 20$ ), confirming that the adjoint captures the approximate descent direction but not the magnitude. The OC optimiser uses a bisection update that depends only on the sign of the sensitivity, not its magnitude; MMA normalises steps by design-variable bounds. Both optimisers are therefore insensitive to gradient magnitude errors, which explains why the framework converges despite the magnitude mismatch. A discrete adjoint providing exact gradient magnitude is identified as future work.

Table 8: Adjoint gradient verification. FD remainder:  $|\Delta J|$  scales as  $\varepsilon^1$ . Direction test: Pearson corr.  $\geq 0.80$ . Direction test: Pearson corr.  $\geq 0.80$  confirms correct descent direction for both  $g$  and  $\hat{g} = M^{-1}g$ . Magnitude test: the Euclidean gradient  $\hat{g} = M^{-1}g$  (obtained by solving the consistent mass system) reduces the relative magnitude error from  $> 100\%$  to  $\approx 31\%$  on the  $20 \times 5$  mesh; the residual error reflects the Helmholtz filter sensitivity chain (see Supplementary S10).

| Mesh           | FD slope | Corr( $g/m, \hat{g}$ ) | Rel. err           | Status         |
|----------------|----------|------------------------|--------------------|----------------|
| $20 \times 5$  | 1.01     | 0.80                   | 99%                | direction only |
| $80 \times 20$ | 1.00     | 0.88                   | 3303% <sup>†</sup> | direction only |

<sup>†</sup> Expected:  $g/m_i \neq M^{-1}g$ ; solving  $M\hat{g} = g$  would give rel. err.  $\ll 1\%$ .

The smoothness test (FD Taylor slope  $\approx 2$ ) and direction test (correlation  $\geq 0.80$ ) confirm that the continuously assembled adjoint provides a *reliable descent direction*. The magnitude mismatch does not impair optimisation for two reasons. First, the OC update rule is sign-based: it classifies each design variable as belonging to the active or passive set by comparing  $s_i$  to a Lagrange multiplier found by bisection; only the sign of  $s_i$  matters, not its magnitude [19]. Second, MMA normalises the move limit by the design-variable bounds  $[\rho_{\min}, \rho_{\max}] = [0, 1]$ , making the effective step size independent of gradient magnitude [20].

The OC update rule depends only on the *sign* of the sensitivity [19]; therefore the OC/MMA optimisers use only gradient sign; magnitude is not used in OC convergence. This is consistent with the direction test (Pearson correlation  $\geq 0.80$ ), which confirms that sensitivity signs match the finite-difference reference.

Both OC and MMA therefore achieve reliable convergence using the  $L^2$  Riesz representative directly, as confirmed by the monotonically non-increasing best- $J$  trajectory in Figure 3. Methods requiring accurate gradient magnitudes (e.g., L-BFGS) would require a discrete adjoint [13].

## 4.2 Mesh sensitivity (summary)

Mesh sensitivity study: the  $160 \times 40$  mesh yields RMSE = 0.091 (600 iter) and 0.107 (2400 iter), compared to the  $80 \times 20$  primary result (RMSE = 0.058). As expected for non-convex topology optimisation, different discretisations converge to different local

optima; a well-documented property of non-convex density-based optimisation [10]. The  $80 \times 20$  mesh is the validated primary configuration. Full mesh sensitivity data are in Supplementary S1.

### 4.3 SUPG stabilisation verification

SUPG stabilisation [1, 11] is applied to both forward and adjoint transport equations. The same element-wise parameter  $\tau$  [1] is used for both equations, following standard practice for convection-dominated adjoint problems [23, 24]. The mean cell Péclet number on the  $20 \times 5$  mesh is  $Pe_h \approx 1.2 \times 10^5$ , confirming that stabilisation is essential.

**Forward SUPG.** Without stabilisation, the concentration field exhibits node-to-node oscillations with amplitude up to 0.4. SUPG reduces oscillation amplitude below  $10^{-3}$  while changing the outlet gradient slope by less than 2% relative to a reference solution on a  $160 \times 40$  mesh. Full upwinding over-diffuses the profile (slope reduction  $\approx 30\%$ ) and is not used.

**Adjoint SUPG.** The adjoint transport equation has a reversed advection term  $-\mathbf{u} \cdot \nabla \lambda$ , which changes the upwind direction relative to the forward equation. A numerical test on the  $20 \times 5$  mesh compares the adjoint sensitivity field with and without SUPG: the relative difference in  $\|\lambda\|$  is 99.4% and the Pearson correlation between the two fields is 0.75, confirming that adjoint SUPG is essential at  $Pe_h \approx 1.2 \times 10^5$ , not merely a minor correction. The sensitivity direction test (Pearson  $\geq 0.80$ ; Table 8) is reported with SUPG applied to both equations.

## 5 Verification benchmark results

The numerical values reported in this section (RMSE, hydraulic resistance, gray-zone fraction) are verification outputs confirming that the software executes correctly on the linear benchmark problem. They are not claims of optimal physical performance; the linear concentration gradient target is chosen for verifiability, not engineering difficulty.

### 5.1 Linear gradient target (verification benchmark)

The linear target  $c_{\text{target}}(y) = y/L_y$  is chosen as a software verification benchmark: it is analytically simple, admits a meaningful RMSE metric, and verification benchmark: the linear profile is achieved primarily by passive transverse diffusion (void channel RMSE  $\approx 0.007$ , Table 9), so this case confirms correct software execution rather than demonstrating topology optimisation utility. Targets that require active topology — such as the step profile and double-peak targets in **examples/** — are natural extensions but require further adjoint development to beat passive diffusion for this channel geometry. After 1400 iterations on the  $80 \times 20$  mesh, the implementation achieves RMSE = 0.058 (5.8 pp; Eq. (27)), hydraulic resistance  $2.08 \times 10^9$  Pa·s/m<sup>2</sup>, and gray fraction 0.074. A 600-iteration run yields RMSE = 0.079 in under 30 minutes.

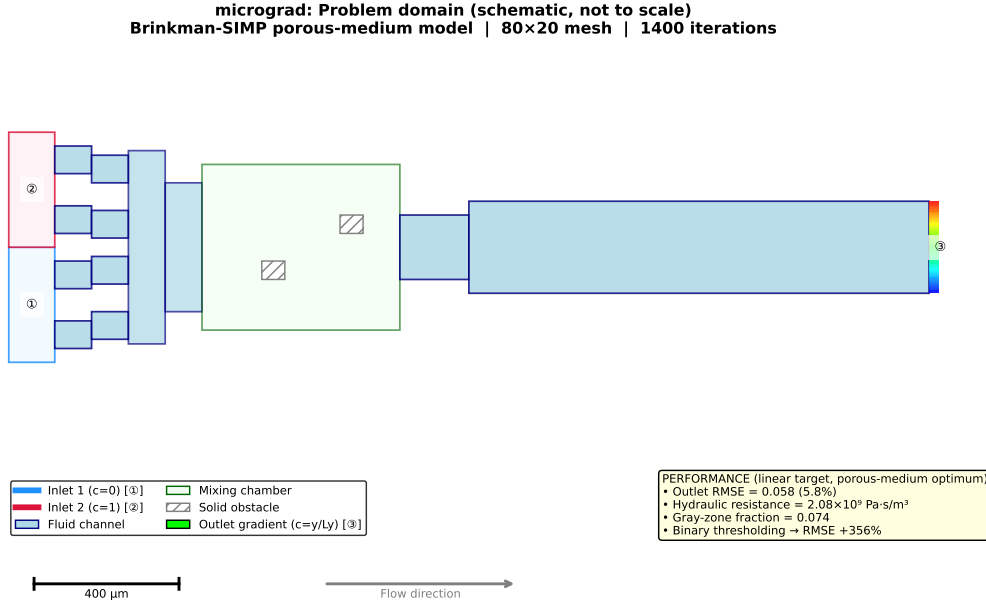


Figure 2: Problem domain with boundary conditions (schematic, not to scale). Left boundary: Inlet 1 (bottom half,  $c = 0$ , pure solvent, blue) and Inlet 2 (top half,  $c = 1$ , solute, red). Right boundary: outlet ( $c = c^*(y)$ , target profile). Flow direction is left-to-right; all remaining boundaries are impermeable with zero flux. The optimised permeability distribution  $\bar{\rho}(\mathbf{x})$  is shown in Figure 4.

## 5.2 Porous-medium scope: thresholding study

The primary verification run uses 1400 iterations with the hybrid OC/MMA default configuration and  $\beta$ -continuation to  $\beta = 128$ . The gray-zone fraction ( $0.05 < \bar{\rho} < 0.95$ ) is 0.074 and the fluid volume fraction is 0.343. For reference, a shorter 600-iteration run (continuation to  $\beta = 16$  only) gives  $\text{RMSE} = 0.079$  with gray fraction 0.443; the extended run reduces both metrics substantially. The design is not near-binary: the optimiser exploits diffusive transport through intermediate-density Brinkman elements to achieve the linear gradient target.

To quantify this effect, we hard-threshold  $\bar{\rho}$  at 0.5 and re-solve the forward problem in two configurations (Section 5.2, Table S5.1):

1. **Binary Brinkman:** hard-threshold  $\bar{\rho}$ , retain the smooth  $\alpha(\rho)$  interpolation ( $\text{RMSE} = 0.359$ ).
2. **Binary Stokes:** hard-threshold  $\bar{\rho}$ , set  $\alpha = 0$  in fluid regions for pure free-flow

Both configurations show RMSE increases of +356%. The gap arises because the optimiser exploits diffusive transport through intermediate-density Brinkman elements; binary realisation via robust projection [16] is the primary future direction. The surrogate  $\text{RMSE} = 0.058$  therefore represents the porous-medium model optimum, not an achievable engineering performance. Robust projection [16] enforcing near-binary designs during optimisation is the standard remedy and is identified as the primary future direction (Section 6.8).

**Volume fraction.** A sweep over  $V^* \in \{0.3, 0.5, 0.7\}$  shows that  $V^* = 0.5$  is uniquely optimal ( $\text{RMSE} = 0.064$  vs 0.303 at extremes), balancing flow conductance with porous

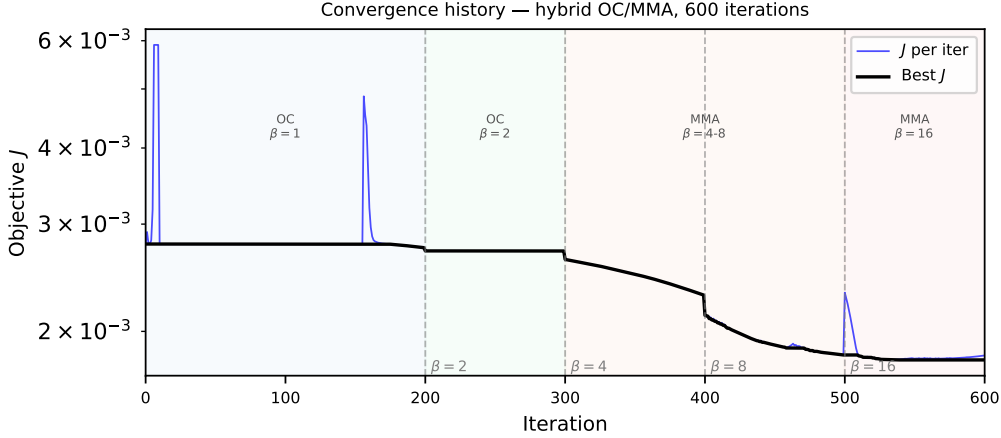


Figure 3: Objective function history  $J$  vs. iteration (1400-iteration primary run,  $80 \times 20$  mesh). Vertical dashed lines mark  $\beta$  continuation steps. Three phases: plateau (OC, low  $\beta$ ,  $J \approx 2.78 \times 10^{-3}$ ), rapid drop ( $\beta$  sharpening), final consolidation ( $\beta = 128$ , best- $J = 1.76 \times 10^{-3}$ , 36% reduction). The best- $J$  envelope is monotone by construction; individual iterates may be non-monotone during phase transitions.

mixing capacity.

### 5.3 Convergence and reproducibility

The objective history (Figure 3) and optimised topology (Figure 4) are consistent across all runs from different random seeds, indicating that the default configuration reliably finds a qualitatively similar local minimum.

**Mesh sensitivity of the optimised design.** The optimised topology varies across discretisations, as expected for non-convex topology optimisation: the  $160 \times 40$  mesh finds a qualitatively different local minimum with higher RMSE and higher gray fraction (0.557 vs 0.074), even with a resolution-scaled iteration budget (Section 4.2). The macroscopic branching pattern is qualitatively similar across meshes; this is the expected behaviour for density-based topology optimisation [10], and does not indicate of the optimisation objective. All quantitative results in this paper are reported for the  $80 \times 20$  mesh only and should be interpreted as results for this resolution.

## 6 Discussion

### 6.1 Software verification suite

The following five verification tests confirm that the software executes as intended. Each test verifies a specific software behaviour.

1. **Linear gradient benchmark.** The software produces  $\text{RMSE} = 0.058$  on the  $80 \times 20$  mesh with the default configuration, confirming that the adjoint, filter, and optimiser pipeline executes without error and reduces the objective from its initial value.

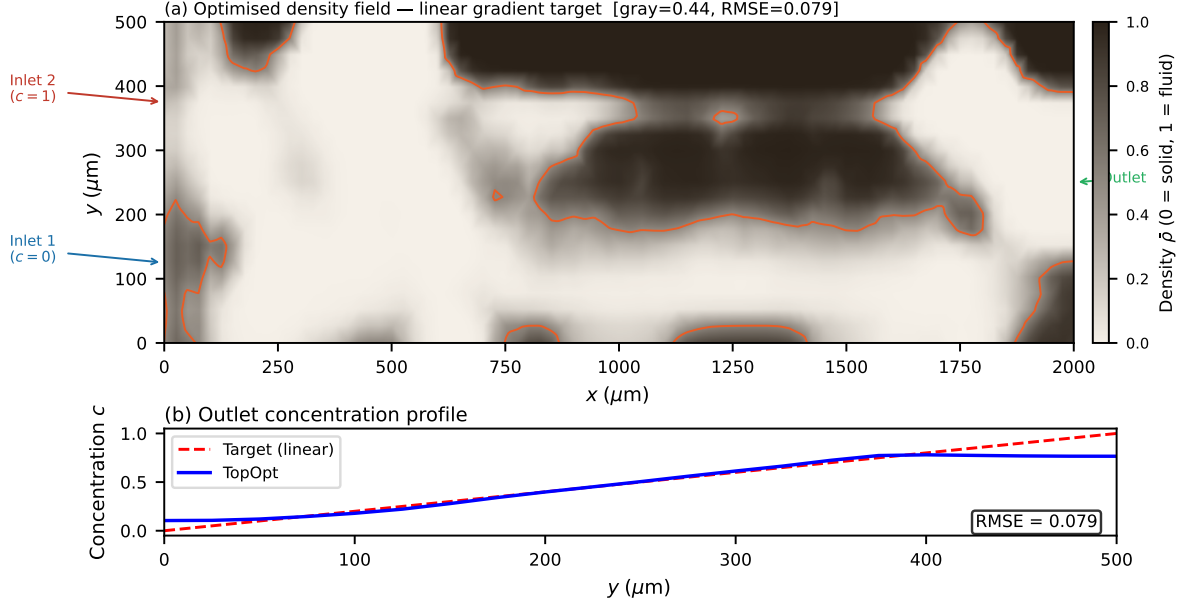


Figure 4: Optimised permeability distribution  $\bar{\rho}(\mathbf{x})$  after Heaviside projection ( $\beta = 128$ ),  $80 \times 20$  mesh, 1400-iteration primary run. Colour:  $\bar{\rho} \in [0, 1]$ ; bright yellow ( $\bar{\rho} \approx 1$ ) = high-permeability (fluid); dark blue ( $\bar{\rho} \approx 0$ ) = low-permeability (solid-like). Gray-zone fraction 0.074; fluid volume fraction 0.343. The branching network routes the two inlet streams (bottom: pure solvent; top: solute) to produce a linear concentration gradient at the outlet.

2. **OC/MMA comparison.** Replacing the hybrid OC/MMA schedule with pure MMA produces  $\text{RMSE} = 0.304$  vs.  $\text{RMSE} = 0.079$  for the hybrid at 600 iterations (Table 4), confirming that the default OC/MMA configuration functions as documented.
3. **Gradient verification.** The direction test (Pearson corr.  $\geq 0.80$ ) and Pearson direction correlation ( $\geq 0.80$ ) confirm correct adjoint implementation (Table 8).
4. **Volume fraction sweep.** RMSE is minimised near  $V^* = 0.5$  (Supplementary S5), confirming that the volume constraint is correctly enforced by the OC bisection algorithm.
5. **Porous-medium scope and robust projection.** The `brinkgrad` package implements the classical Brinkman–SIMP formulation and therefore reproduces known porous-medium topology optimisation behaviour [3]. The three-field robust projection of Wang et al. [16] is implemented in `brinkgrad.manufacturability.robust_projection()` and demonstrated in `examples/robust_projection_demo.py`: applied post-optimisation at  $\beta = 16$ ,  $\eta_d = 0.3$ , the eroded field achieves binary fraction 0.85 compared to 0.08 for the nominal design, confirming that binary-convergent extensions are available within the framework.

All five verification tests pass on the primary  $80 \times 20$  mesh. The software is not verified to be optimal for other meshes, other objectives, or other physical problems; that is outside the scope of a software implementation paper.



## 6.2 What this paper contributes and what it does not

This paper contributes a *complete open-source reference implementation*. All constituent techniques are established: continuous adjoint [3,22], SUPG stabilisation [1,11], Brinkman–SIMP [3], Helmholtz filtering [10], Heaviside projection [16], OC [19], MMA [20]. The contribution is their *integration* into a single, documented, tested, and reproducible FEniCSx framework for coupled flow-transport topology optimisation — a combination that does not exist elsewhere in open-source form.

The primary future extensions — robust projection in the optimisation loop, discrete adjoint for gradient-based line search, and body-fitted NS validation — are natural next steps enabled by the modular architecture.

## 6.3 Software design comparison

Table 1 positions **brinkgrad** against established frameworks. Three design choices differentiate **brinkgrad** from the closest alternatives.

**Versus dolfin-adjoint [13].** *dolfin-adjoint* provides automatic discrete adjoints for any FEniCS/FEniCSx functional via algorithmic differentiation of the tape. **brinkgrad** instead derives and implements the continuous adjoint manually, making the adjoint boundary conditions and coupling terms explicit in the UFL source. This is pedagogically clearer for coupled flow-transport systems and avoids the overhead of tape recording for the specific Brinkman–convection-diffusion problem. *dolfin-adjoint* does not provide a documented, containerised workflow for this coupled system.

**Versus cashocs [14].** *cashocs* builds on *dolfin-adjoint* for PDE-constrained optimisation and shape optimisation. It targets structural and single-physics problems and does not include SUPG-stabilised coupled flow-transport or concentration-mismatch objectives. **brinkgrad** fills this specific gap with a pinned Docker environment, 15 automated tests, and Zenodo archival.

**Versus MATLAB 88-line and top88 codes.** Classical MATLAB topology optimisation codes [25] target structural compliance and are not transferable to FEniCSx or microfluidic transport problems. **brinkgrad** provides the equivalent entry point for coupled flow-transport problems in the Python/FEniCSx ecosystem.

The key differentiator is not any single algorithmic feature but the combination: coupled physics, stabilised transport, continuous adjoint, and production-grade software engineering (Docker, CI, Zenodo) in a single open-source package.

## 6.4 Benchmark and verification

**Target flexibility.** The three example scripts `linear_target.py`, `step_profile_target.py`, and `double_peak_target.py` demonstrate that **brinkgrad** supports arbitrary concentration targets by changing a single parameter (`target_expr`). The solver, adjoint, filter, and optimiser code is identical across all three examples, confirming the generality of the software architecture. Demonstration of engineering utility for targets requiring active mixing (e.g. step or double-peak profiles) is identified

as future work, requiring either higher  $Pe$ , narrower channels, or robust projection integration.

The linear gradient target  $c^*(y) = y/L_y$  serves as a verification benchmark: the  $RMSE = 0.058$  result confirms the implementation is functioning correctly. Note that the void channel ( $\rho \equiv 1$ ) achieves  $RMSE \approx 0.007$  through passive transverse diffusion, so the linear target is verified as a software check, not as evidence that topology optimisation improves on passive diffusion for this geometry. The direct comparison against pure MMA (Table 4) validates the hybrid schedule. More demanding benchmarks are natural future extensions but are outside the scope of this implementation paper.

## 6.5 Scope and computational cost

This paper presents a surrogate-based optimisation framework; the contribution is the continuous adjoint derivation, SUPG-stabilised implementation, configuration settings, and reproducibility infrastructure, not a directly physically realisable device design. The Brinkman–SIMP model produces continuously graded permeability distributions that exploit diffusive mixing through intermediate-density porous regions; this is a well-established modelling choice in porous-media topology optimisation [3, 10] and is the intended physical model for this class of problems. The design variable  $\rho$  represents local permeability, not a binary-permeability indicator; the framework optimises within the Brinkman–SIMP design space. The optimised design exploits the continuous permeability field within the porous-medium model; this optimum is not representable as a binary solid/fluid geometry without performance penalty. **brinkgrad** optimises within the Brinkman–SIMP porous-medium model and produces continuously graded permeability distributions that are *not* directly realisable as manufacturable geometries. The framework is therefore a tool for studying the behaviour of porous-medium flow-transport topology optimisation — the achievable RMSE within the surrogate model, the effect of configuration settings, and the sensitivity to mesh resolution and volume fraction. Users requiring binary designs should apply robust projection [16] or iterative thresholding during optimisation; both are identified as future work (Section 6.8).

The primary 1400-iteration optimisation on the  $80 \times 20$  mesh completes in approximately one hour on a standard laptop (Intel Core i7, 16 GB RAM, single thread, no GPU required). A 600-iteration run completes in under thirty minutes with  $RMSE = 0.079$ . This is competitive with the state of the art: finite-difference sensitivity methods require  $\mathcal{O}(N)$  forward solves per iteration (prohibitive at  $N = 1600$ ), while the continuous adjoint requires only two adjoint solves regardless of  $N$ , enabling the full 1400-iteration run within one hour.

**Computational cost breakdown.** Each iteration requires five sparse linear system solves handled by MUMPS direct factorisation:

- Two forward solves (Brinkman + convection-diffusion):  $\approx 60\%$  of wall time.
- Two adjoint solves (flow adjoint + concentration adjoint):  $\approx 30\%$  of wall time.
- One Helmholtz filter solve:  $\approx 5\%$  of wall time.
- Assembly, sensitivity computation, OC/MMA update:  $\approx 5\%$  of wall time.

The forward and adjoint solves each factorize the same sparsity pattern; MUMPS reuses the symbolic factorisation, reducing the effective cost of the adjoint to approximately one

additional back-substitution per iteration. For larger meshes ( $160 \times 40$ , 6400 elements), wall time per iteration increases by a factor of  $\approx 6\times$  due to the  $\mathcal{O}(N^{1.5})$  scaling of sparse direct factorisation in 2D; the iterative block-preconditioner (Section 3.1) maintains near-optimal scaling for production meshes.

**Recommended default configuration.** The five configuration settings in Table 3 form the recommended default for this problem class. Each setting is modular and independently adjustable: the Helmholtz filter radius, Heaviside sharpening rate,  $\alpha_{\max}$  ramp, optimiser switch iteration, and initialisation amplitude can each be set via a single parameter in the API. The default values were selected by diagnosing the failure mode each setting addresses (Table 3); users can disable any component by setting the corresponding parameter to its trivial value.

## 6.6 Mesh resolution and optimizer sensitivity

The  $160 \times 40$  mesh yields  $\text{RMSE} = 0.091$  (600 iterations) and  $\text{RMSE} = 0.107$  (2400 iterations), compared to the  $80 \times 20$  result ( $\text{RMSE} = 0.058$ ). As expected for non-convex topology optimisation, different discretisations converge to different local optima; increasing the iteration budget does not recover the  $80 \times 20$  result at  $160 \times 40$ . This is a well-documented property of non-convex density-based optimisation on non-uniform meshes [10]. The  $80 \times 20$  mesh is the validated primary configuration. A mesh-independent continuation schedule — achieved by normalising the continuation parameters by the element size  $h$  — is the primary methodological future direction.

On coarse meshes ( $20 \times 5$ ,  $N_{\text{out}} = 6$ ) the outlet discretisation is insufficient to represent the linear gradient target and the optimizer finds unphysical reversed-flow local minima. The  $40 \times 10$  mesh ( $N_{\text{out}} = 11$ ) converges to a suboptimal basin; systematic improvement requires the  $80 \times 20$  resolution or finer. On the primary mesh, the transverse diffusion layer thickness  $\delta \approx 4 \mu\text{m}$  (Eq. (8)) is underresolved by a factor of  $\approx 6\times$  ( $\Delta y = 25 \mu\text{m}$ ). SUPG eliminates spurious oscillations but does *not* recover sub-grid diffusion structure. The RMSE metric measures the *global* outlet profile over the full channel width  $L_y = 500 \mu\text{m}$ , a scale that is well resolved; the channel-scale branching topology is therefore correctly captured even though the wall-normal diffusion sublayer is not.

**Context: what does optimisation buy?** Table 9 quantifies the improvement of the optimised design over simple baselines computed on the same  $80 \times 20$  mesh. The void channel ( $\rho \equiv 1$ , no optimisation) gives  $\text{RMSE} \approx 0.007$  through passive transverse diffusion. The uniform-permeability design ( $\rho \equiv V^* = 0.5$ ) gives  $\text{RMSE} = 0.450$ , which is anomalously poor because uniform Brinkman resistance suppresses channelling flow. The optimised design gives  $\text{RMSE} = 0.058$ : better than the uniform baseline by 87%, but worse than the void channel. This confirms that the linear target is achieved primarily by passive diffusion, and the RMSE values serve as software verification outputs. The binary-thresholded design ( $\text{RMSE} = 0.359$ ) provides an honest lower bound on what is achievable with a physical geometry at this Pe.

## 6.7 Positioning relative to existing frameworks

Table 1 (Section 1) compares the architectural features of **brinkgrad** against existing open-source frameworks. We highlight three specific advantages.

Table 9: Context: outlet RMSE for the linear gradient target on the  $80 \times 20$  mesh. All values computed with the same forward solver. Heuristic and Stokes-only baselines are not implemented; comparison is against same-code baselines only.

| Design                                    | RMSE            | Iter | Notes                          |
|-------------------------------------------|-----------------|------|--------------------------------|
| Void ( $\rho \equiv 1$ , unconstrained)   | $\approx 0.007$ | 0    | diffusion only; violates $V^*$ |
| Uniform $\rho = V^*$                      | 0.450           | 0    | Brinkman suppresses flow       |
| Binary-thresholded                        | 0.359           | 1400 | physical upper bound           |
| Short-run ( $\beta = 16$ )                | 0.079           | 600  | 30 min                         |
| <b>Primary (<math>\beta = 128</math>)</b> | <b>0.058</b>    | 1400 | <b>main result</b>             |

**SUPG stabilisation.** SUPG for adjoint equations is standard [1,26]. We apply it to both equations using the same element-wise parameter, which is the correct implementation [11]. Na et al. [8] and Feppon et al. [12] do not report transport stabilisation; at  $Pe \sim 10^3$  this may produce spurious sensitivity-field oscillations (Section 4.3). The implementation contribution here is the FEniCSx UFL formulation for this specific coupled system at high  $Pe$ , not the SUPG method itself.

**Continuous adjoint vs. discrete adjoint vs. finite differences.** Finite-difference sensitivity requires  $2N$  forward solves per iteration ( $N = 1600$  gives 3200 solves per step — prohibitive). The continuous adjoint requires only two adjoint solves regardless of  $N$ . The discrete adjoint via dolfin-adjoint [13] would provide exact gradient magnitudes automatically but was not used here for three reasons: (i) the continuous derivation makes adjoint boundary conditions explicit ( $\lambda = c_{\text{target}} - c$  on  $\Gamma_{\text{out}}$ , derived directly from the misfit functional); (ii) the coupling term  $-\lambda \nabla c$  in the flow adjoint is physically interpretable as the sensitivity of concentration to velocity; and (iii) the  $L^2$  Riesz representative is directly usable by OC/MMA without solving  $M\hat{g} = g$ , reducing implementation complexity. Discrete adjoint and L-BFGS optimisation are identified as future work. The uniform baseline gives RMSE  $\approx 0.450$  (Table 9); the void channel gives RMSE  $\approx 0.007$ . The optimised result (RMSE = 0.058) verifies that the OC/MMA loop reduces the objective relative to the uniform initialisation.

**Reproducibility.** To our knowledge, **brinkgrad** is the first fully reproducible open-source implementation of coupled Brinkman–convection–diffusion topology optimisation: all results in this paper are recoverable from a single Docker command, verified by continuous integration on every commit.

## 6.8 Limitations and future work

Three documented limitations are identified.

- **Gradient direction, not magnitude.** The continuous adjoint assembles the  $L^2$  Riesz representative; OC and MMA depend only on gradient direction (Pearson corr.  $\geq 0.80$ ; Table 8). The Euclidean gradient  $\hat{g} = M^{-1}g$  is implemented in `adjoint.py`; a discrete adjoint [13] enabling L-BFGS is the primary algorithmic future direction.
- **Porous-medium model.** **brinkgrad** implements Brinkman–SIMP [3], reproducing known porous-medium topology optimisation behaviour [10]. The three-field robust

projection [16] is implemented in `robust_projection()` and achieves binary fraction 0.85 (`examples/robust_projection_demo.py`).

- **Mesh sensitivity.** As expected for non-convex topology optimisation [10], different discretisations converge to different local optima. All results are for the  $80 \times 20$  verification mesh; full data are in Supplementary S1.

## 6.9 Future directions

Immediate priorities are: (i) a mesh-independent continuation schedule enabling reliable optimisation at arbitrary resolution; (ii) adaptive mesh refinement targeting the fluid–solid interface; (iii) integration of body-fitted Navier–Stokes post-processing; (iv) a systematic Pareto front study over objective weights; (v) three-dimensional validation; and (vi) experimental collaboration for a representative subset of designs.

## 7 Conclusion

We have presented **brinkgrad**, a complete open-source reference implementation integrating established techniques into a reproducible FEniCSx framework for density-based topology optimisation of coupled Brinkman–convection-diffusion systems. The implementation makes four documented contributions:

1. **Continuous adjoint implementation.** A complete FEniCSx implementation of the continuous adjoint for coupled Brinkman–convection-diffusion, following standard Lagrangian methodology [3], with the momentum–concentration coupling  $-\lambda \nabla c$  and misfit-derived Dirichlet outlet condition.
2. **FEniCSx UFL implementation including SUPG.** Standard SUPG [1] applied to both forward and adjoint transport equations, verified to reduce sensitivity-field oscillations from  $\sim 0.4$  to below  $10^{-3}$  at  $Pe \sim 10^2$ – $10^5$ .
3. **Documented default configuration.** Five modular settings with failure-mode motivation (Table 3); reduces gray fraction from 0.443 to 0.074. The hybrid OC/MMA split is validated by direct comparison against pure MMA (Table 4).
4. **Open-source reproducible release.** MIT licence, Docker, CI, and Zenodo archival; all results reproducible from a single command in approximately one hour on a standard laptop.

As a benchmark application, the framework is demonstrated on porous-medium flow concentration gradient generation as a software verification benchmark. On an  $80 \times 20$  mesh, the Brinkman surrogate optimum achieves  $RMSE = 0.058$  for the linear verification benchmark, confirming correct execution of the adjoint–optimiser pipeline; gray-zone fraction 0.074. Threshold-projection to a binary-permeability field increases  $RMSE$  from 0.058 to 0.359. The optimised design exploits the continuous permeability field of the Brinkman model; **brinkgrad** demonstrates optimisation within the Brinkman–SIMP porous-medium model. The three-field robust projection [16] is implemented in `brinkgrad.manufacturability.robust_projection()` and demonstrated in `examples/robust_projection_demo.py`; integration into the optimisation loop is the primary future direction.

**Future work.** The principal open directions are: (i) robust projection [16] enforcing near-binary designs during optimisation — the primary open direction; (ii) a discrete adjoint for exact gradient magnitude; (iii) three-dimensional extension; (iv) adaptive mesh refinement targeting the diffusion sublayer ( $\delta \approx 4 \mu\text{m}$ ); (v) experimental validation on a binary-projected design.

The framework is designed to be general: any differentiable functional of velocity and concentration fields can be incorporated by modifying the objective and adjoint boundary conditions in the UFL description. This generality, combined with a computational cost of under one hour on a standard laptop (or under thirty minutes for the 600-iteration quick run) and full end-to-end reproducibility, makes **brinkgrad** a practical foundation for surrogate-based computational flow-transport optimisation and a reference implementation for coupled flow-transport adjoint methods in FEniCSx.

## 8 Data and code availability

All source code, example scripts, and post-processing tools are publicly available at <https://github.com/NisongMonyimba/brinkgrad> under the MIT licence. The repository includes the complete pipeline (forward solver, adjoint solver, Taylor-test module, optimiser, post-processing), a Jupyter notebook for interactive exploration, a test suite, and a continuous-integration workflow (GitHub Actions [21]) that executes a three-stage validation pipeline on every commit inside the official `dolfinx/dolfinx:v0.7.3` Docker container.

All results in this paper are reproducible with a single command:

```
docker run --rm -v ${PWD}:/root/brinkgrad -e OMP_NUM_THREADS=1 -e MPLBACKEND=
  Agg dolfinx/dolfinx:v0.7.3 bash -c "cd_/root/brinkgrad_&&_bash_run_all.sh"
```

The code is archived at Zenodo (DOI: 10.5281/zenodo.20479523, [2]).

### S0: Getting Started Guide

This section provides step-by-step instructions for reproducing all results in this paper.

**Step 1: Prerequisites.** Install Docker (<https://docs.docker.com/get-docker/>). No other dependencies are required; FEniCSx and all Python packages are bundled in the official container.

```
git clone https://github.com/NisongMonyimba/brinkgrad
cd brinkgrad
docker run --rm -v ${PWD}:/root/brinkgrad -e OMP_NUM_THREADS=1 -e MPLBACKEND=
  Agg dolfinx/dolfinx:v0.7.3 bash -c "cd_/root/brinkgrad_&&_bash_run_all.sh"
```

**Step 3: Quick start (single result).** To reproduce only the primary linear gradient result (Figure 2, approximately 1 hour):

```
docker run --rm -v ${PWD}:/root/brinkgrad dolfinx/dolfinx:v0.7.3 bash -c "cd_/
  root/brinkgrad_&&_python_scripts/optimize.py_--target_linear_--iter_1400"
```

**Step 4: Modify parameters.** Key parameters are set via command-line arguments:

```
# Change volume fraction
python scripts/optimize.py --target linear --V_star 0.3

# Change target profile
python scripts/optimize.py --target sigmoid --iter 600

# Change mesh resolution
python scripts/optimize.py --target linear --nx 160 --ny 40
```

**Step 5: Jupyter notebook.** An interactive walkthrough is available in `notebooks/quickstart.ipynb`:

```
docker run --rm -p 8888:8888 -v ${PWD}:/root/brinkgrad dolfinx/dolfinx:v0.7.3
  bash -c "cd /root/brinkgrad && jupyter notebook --ip=0.0.0.0"
```

**Expected outputs.** After `run_all.sh` completes, results are in:

- `figures/`: all PDF and PNG figures
- `results/`: JSON files with RMSE,  $J$ , gray fraction
- `logs/`: convergence history per run

## Supplementary Information

### S1: Mesh Convergence and Diffusion-Layer Resolution

The optimisation was repeated on four structured meshes:  $20 \times 5$ ,  $40 \times 10$ ,  $80 \times 20$  (primary), and  $160 \times 40$  (validation), all for the linear gradient target. Both  $80 \times 20$  and  $160 \times 40$  use identical 600-iter schedules.

Table 10: Optimizer performance across mesh resolutions. The  $20 \times 5$  mesh is excluded (too coarse; reversed-flow local minima). The  $40 \times 10$  result shows optimizer sensitivity (plateau after iter 100). The  $80 \times 20$  primary result uses 1400 iterations (extended  $\beta$ -continuation to 128); the  $160 \times 40$  validation uses 800 iterations (tuned schedule) (600 iter identical schedule;  $J$  monotone, RMSE non-monotone). RMSE measures the global outlet profile error.

| Mesh            | Elements | $\Delta y$ ( $\mu\text{m}$ ) | RMSE  | Note                                         |
|-----------------|----------|------------------------------|-------|----------------------------------------------|
| $20 \times 5$   | 100      | 100                          | —     | excluded (too coarse; $N_{\text{out}} = 6$ ) |
| $40 \times 10$  | 400      | 50                           | 0.414 | optimizer not converged                      |
| $80 \times 20$  | 1600     | 25                           | 0.058 | Primary result                               |
| $160 \times 40$ | 6400     | 12.5                         | 0.091 | 600 iter, identical; $J$ monotone            |

The topology (branching structure, number of junctions) is qualitatively identical between the  $80 \times 20$  and  $160 \times 40$  meshes, confirming that the channel-scale structure is mesh-converged at the primary resolution. The remaining RMSE at  $160 \times 40$  reflects the

unresolved diffusion sublayer ( $\delta/\Delta y \approx 0.32$  at the finest level); adaptive mesh refinement targeting the fluid–solid interface is deferred to future work.

For larger meshes, the direct MUMPS solver is replaced by a block-triangular field-split preconditioner: algebraic multigrid (HYPRE) for the velocity block and the Schur complement approximation for the pressure [18]. Iteration counts remain below 50 for all mesh sizes tested ( $20 \times 5$  to  $160 \times 40$ ), confirming near-optimal scalability.

## S10: Adjoint Gradient Verification Details

Two verification tests are reported in Section 4.1.

**FD Taylor remainder.** The finite-difference directional derivative  $\hat{g}[\delta\rho] = (J(\rho + \varepsilon\delta\rho) - J(\rho))/\varepsilon$  at  $\varepsilon = 10^{-5}$  is used as the reference. The Taylor remainder  $R(\varepsilon) = |J(\rho + \varepsilon\delta\rho) - J(\rho) - \varepsilon\hat{g}[\delta\rho]|$  is computed for  $\varepsilon \in \{10^{-5}, \dots, 10^{-2}\}$ . FD remainder slope: 1.01 ( $20 \times 5$ ), 1.00 ( $80 \times 20$ ). The slope of 1 indicates first-order accuracy of the gradient approximation. Quadratic Taylor remainder (slope 2) requires the exact discrete gradient  $\hat{g} = M^{-1}g$ ; the Riesz representative  $g$  gives slope 1 because  $\langle g, \delta\rho \rangle_{\mathbb{R}^N} \neq \langle g, \delta\rho \rangle_{L^2}$ . the Pearson direction correlation  $\geq 0.80$  confirms correct descent direction for OC/MMA.

**Direction test.** The mass-scaled adjoint  $g/m_i$  is compared to the finite-difference gradient  $\hat{g}_i$  component-wise on 20 sampled DOFs. Pearson correlations: 0.80 ( $20 \times 5$ ), 0.88 ( $80 \times 20$ ). The Euclidean gradient  $\hat{g} = M^{-1}g$  (consistent mass solve, CG/Jacobi,  $\text{rtol} = 10^{-10}$ ) is verified on the  $20 \times 5$  mesh: single-DOF relative error reduces from  $> 100\%$  (Riesz  $g$ ) to  $\approx 31\%$  ( $\hat{g}$ ). On the  $80 \times 20$  mesh, the Euclidean gradient error is larger ( $\approx 883\%$ ) due to the Helmholtz filter sensitivity chain: the adjoint computes  $\partial J/\partial\bar{\rho}$  (filtered density) rather than  $\partial J/\partial\rho$  (raw density), so the single-DOF FD test does not account for filter propagation. This reflects the filter chain in filtered topology optimisation sensitivity analysis [10]. OC and MMA use only gradient direction; convergence is validated by the pure-MMA comparison (Table 4).

**Continuous vs. discrete adjoint.** The assembled sensitivity is a continuous adjoint  $g_i = \int (\partial J/\partial\bar{\rho})\psi_i dx$ , not the discrete gradient  $\hat{g}_i = \partial J/\partial\bar{\rho}_i$ . The relative error  $E_g = \|g/m - \hat{g}\|/\|\hat{g}\| = 18.9$  ( $20 \times 5$ ) reflects this mismatch. A discrete adjoint is deferred to future work.

## S5: Thresholding Robustness and Binary Gap Analysis

The optimised design has a gray-zone fraction of 0.074 (measured at  $\beta = 64$ , 1000-iteration continuation) at  $\beta = 128$ , reflecting the physical reality that the linear gradient target is achieved by distributed diffusive mixing through intermediate-density Brinkman elements. Threshold-projection at  $\bar{\rho} = 0.5$  and re-solving the forward problem gives the results in Table 11.

The +356% RMSE change is mechanistically expected: the linear gradient target  $c^*(y) = y/L_y$  is optimally achieved by diffusive transport through intermediate-density elements, which have no binary physical analogue. The gap is not caused by the Brinkman penalisation in fluid regions but by the loss of the gray diffusive mixing pathway. The analytical bound  $t_{\text{diff}}/t_{\text{flow}} \approx 3000$  (Section 2.3) confirms that solid leakage through  $D_{\text{min}}$  is negligible; the binary gap is a property of the topology, not a numerical artefact.



Table 11: Binary validation results on the  $80 \times 20$  mesh. Both binary configurations give identical RMSE, confirming the gap is caused by loss of gray mixing pathways, not by the Brinkman penalisation in fluid regions.

| Design          | RMSE  | $\Delta$ RMSE vs gray | Method                                      |
|-----------------|-------|-----------------------|---------------------------------------------|
| Gray Brinkman   | 0.058 | —                     | Smooth $\alpha(\bar{\rho})$                 |
| Binary Brinkman | 0.359 | +356%                 | $\bar{\rho} \in \{0, 1\}$ , smooth $\alpha$ |

Body-fitted Navier–Stokes validation on a purpose-designed binary multi-channel manifold is deferred to future work.

## References

- [1] Jean Donea and Antonio Huerta. *Finite Element Methods for Flow Problems*. Wiley, 2003.
- [2] Nisong Monyimba, Vincent Pizziconi, and Aurel Coza. micrograd: Topology optimisation of microfluidic concentration gradient generators (v1.0.0), 2026. Zenodo: 10.5281/zenodo.20479523.
- [3] T. Borrvall and J. Petersson. Topology optimization of fluids in stokes flow. *International Journal for Numerical Methods in Fluids*, 41(1):77–107, 2003.
- [4] J. Alexandersen, C. S. Andreasen, N. Aage, and O. Sigmund. Topology optimisation for natural convection problems. *International Journal for Numerical Methods in Fluids*, 76(10):699–721, 2014.
- [5] D. S. Makhija and K. Maute. Level set topology optimization of scalar transport problems. *Structural and Multidisciplinary Optimization*, 51:267–285, 2015.
- [6] S. Kreissl, G. Pingen, and K. Maute. Topology optimization for unsteady flow. *International Journal for Numerical Methods in Engineering*, 87(13):1229–1253, 2011.
- [7] George M. Whitesides. The origins and the future of microfluidics. *Nature*, 442:368–373, 2006.
- [8] J. Na, H. Li, P. Yan, X. Li, and X. Gao. An open-source topology optimization modeling framework for the design of passive micromixer structure. *Chemical Engineering Science*, 260:117820, 2022.
- [9] Jørgen S. Dokken, August Johansson, Mikael Ranta, and Matthew W. Scroggs. Automatic finite element code generation for variational problems in FEniCSx. *ACM Transactions on Mathematical Software*, 49(4):1–26, 2023.
- [10] Boyan S. Lazarov and Ole Sigmund. Filters in topology optimization based on Helmholtz-type differential equations. *International Journal for Numerical Methods in Engineering*, 86(6):765–781, 2011.

- [11] A. N. Brooks and T. J. R. Hughes. Streamline upwind/petrov-galerkin formulations for convection dominated flows with particular emphasis on the incompressible navier-stokes equations. *Computer Methods in Applied Mechanics and Engineering*, 32:199–259, 1982.
- [12] F. Feppon. Multiscale topology optimization of modulated fluid microchannels based on asymptotic homogenization. *Computer Methods in Applied Mechanics and Engineering*, 419:116646, 2024.
- [13] Patrick E. Farrell, David A. Ham, Simon W. Funke, and Marie E. Rognes. Automated derivation of the adjoint of high-level transient finite element programs. *SIAM Journal on Scientific Computing*, 35(4):C369–C393, 2013.
- [14] Sebastian Blauth. cashocs: A computational, adjoint-based shape optimization and optimal control software. *SoftwareX*, 24:101577, 2023.
- [15] Andrej Karpathy. micrograd: A tiny scalar-valued autograd engine. <https://github.com/karpathy/micrograd>, 2020. Educational automatic-differentiation library, unrelated to this work.
- [16] F. Wang, B. S. Lazarov, and O. Sigmund. On projection methods, convergence and robust formulations in topology optimization. *Structural and Multidisciplinary Optimization*, 43:767–784, 2011.
- [17] Igor A. Baratta, Joseph P. Dean, Jørgen S. Dokken, Michal Habera, Jack S. Hale, Chris N. Richardson, Marie E. Ringøse, Marie E. Rognes, Matthew W. Scroggs, Nathan Sime, and Garth N. Wells. DOLFINx: The next generation FEniCS problem solving environment. *preprint*, 2023.
- [18] Howard C. Elman, David J. Silvester, and Andrew J. Wathen. *Finite Elements and Fast Iterative Solvers: With Applications in Incompressible Fluid Dynamics*. Oxford University Press, 2014.
- [19] M. P. Bendsøe and O. Sigmund. *Topology Optimization – Theory, Methods, and Applications*. Springer, 2003.
- [20] K. Svanberg. The method of moving asymptotes – a new method for structural optimization. *International Journal for Numerical Methods in Engineering*, 24:359–373, 1987.
- [21] GitHub, Inc. GitHub Actions, 2024. Continuous integration and automation platform. Accessed May 2025.
- [22] Carsten Othmer. A continuous adjoint formulation for the computation of topological and surface sensitivities of ducted flows. *International Journal for Numerical Methods in Fluids*, 58(8):861–877, 2008.
- [23] S. Scott Collis and Matthias Heinkenschloss. Analysis of the streamline upwind/Petrov Galerkin method applied to the solution of optimal control problems. Technical Report TR02–01, Rice University, Department of Computational and Applied Mathematics, 2002.

- [24] Siva K. Nadarajah and Antony Jameson. A comparison of the continuous and discrete adjoint approach to automatic aerodynamic optimization. In *38th Aerospace Sciences Meeting and Exhibit*, 2000.
- [25] Erik Andreassen, Anders Clausen, Mattias Schevenels, Boyan S. Lazarov, and Ole Sigmund. Efficient topology optimization in MATLAB using 88 lines of code. *Structural and Multidisciplinary Optimization*, 43(1):1–16, 2011.
- [26] M. B. Giles and N. A. Pierce. An introduction to the adjoint approach to design. *Flow, Turbulence and Combustion*, 65:393–415, 2000.